



Frankfurt University of Applied Sciences

Fachbereich 2: Informatik und Ingenieurwissenschaften

Masterarbeit

Wissenschaftliche Berichtsabgabe zur Masterarbeit

Thema:

„KI-basierte Anomalieerkennung auf ressourcenbeschränkter Edge-Hardware“

Erster Prüfer: Herr Prof. Dr. Karsten Schmidt

Zweite Prüferin: Frau M.Sc. Virginia Kramer

Studiengang: Mechatronik und Robotik

Bearbeiter: Abdulmalik Alzubi

Matrikelnummer: 1224063

Zeitraum: 26.05.2026 – 27.10.2026

Abgabedatum: 27.10.2026

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig erstellt und keine anderen als die angegebenen Hilfsmittel benutzt habe.

Soweit ich auf fremde Materialien, Texte oder Gedankengänge zurückgegriffen habe, enthalten meine Ausführungen vollständige und eindeutige Verweise auf die Urheber und Quellen.

Alle weiteren Inhalte der vorgelegten Arbeit stammen von mir im urheberrechtlichen Sinn, soweit keine Verweise und Zitate erfolgen.

Datum: Frankfurt, den XXXXX

Unterschrift:

A handwritten signature in black ink, appearing to be 'A. Alzubi', with a stylized flourish underneath.

Abdulmalik Alzubi

Abbildungsverzeichnis

Es konnten keine Einträge für ein Abbildungsverzeichnis gefunden werden.

Tabellenverzeichnis

Tabelle 2-1: Kennzahlen der binären Anomalieerkennung.....	7
Tabelle 3-1: Ausgewählte Vorarbeiten und ihre Bedeutung für die Untersuchung.....	10
Tabelle 4-1: Anforderungen, Prioritäten und Nachweise.....	14
Tabelle 5-1: Bewertung der Prüfstandsalternativen.....	18
Tabelle 5-2: Kriterien und Gewichte für den Prüfstand.....	18
Tabelle 5-3: Gewichtete Bewertung der Prüfstandsalternativen.....	19
Tabelle 5-4: Vergleich der Sensorik und Messketten.....	20
Tabelle 5-5: Bewertung der Edge-Plattformen.....	21
Tabelle 5-6: Kriterien und Gewichte für die Edge-Plattform.....	21
Tabelle 5-7: Gewichtete Bewertung der Edge-Plattformen.....	22
Tabelle 5-8: Bewertung der Signalrepräsentationen.....	23
Tabelle 5-9: Auswahl der Anomalieverfahren.....	24
Tabelle 5-10: Kriterienbasierter Vergleich der Softwarealternativen.....	25
Tabelle 5-11: Kriterien und Gewichte für die Softwareauswahl.....	26
Tabelle 5-12: Gewichtete Bewertung der Softwarealternativen.....	26
Tabelle 5-13: Systementscheidungen und ausstehende Nachweise.....	28
Tabelle 6-1: Prüfstandangaben und noch offene Randbedingungen.....	30
Tabelle 6-2: Implementierte Komponenten der Verarbeitung.....	32
Tabelle 6-3: Aktuelle Pilotparameter und ihre Aussagegrenzen.....	33
Tabelle 6-4: Gemeinsame Eingabebasis und implementierte Verfahren.....	35
Tabelle 6-5: Vorhandene Entwicklungsbelege und offene Nachweise.....	38

Inhalt

Erklärung.....	i
Abbildungsverzeichnis.....	ii
Tabellenverzeichnis.....	iii
Inhalt.....	iv
Abkürzungsverzeichnis.....	vi
1 Einleitung, Problemstellung und Zielsetzung.....	1
1.1 Ausgangssituation und Motivation.....	1
1.2 Wissenschaftliche Problemstellung.....	1
1.3 Zielsetzung der Arbeit.....	2
1.4 Forschungsfragen und Hypothesen.....	2
1.5 Abgrenzung der Arbeit.....	3
1.6 Aufbau der Arbeit.....	3
2 Theoretische Grundlagen.....	4
2.1 Zustandsüberwachung und Anomalieerkennung.....	4
2.2 Verfahrensklassen für die Anomalieentscheidung.....	4
2.2.1 Statistische Schwellenwertverfahren.....	4
2.2.2 Klassische Verfahren im Normaldaten-Szenario.....	5
2.2.3 Autoencoder.....	5
2.3 Messdaten und Fensterbildung.....	5
2.4 Datenaufteilung und Vorverarbeitung.....	6
2.5 Schwellenwertkalibrierung.....	6
2.6 Erkennungsqualität und Aussagekraft der Evaluation.....	7
2.7 Lokale Ausführung und Zeitverhalten.....	8
2.8 Veränderte Normalbedingungen.....	8
3 Stand der Technik und verwandte Arbeiten.....	9
3.1 Datenbasis und Messbedingungen.....	9
3.2 Verfahren und Edge-Ausführung.....	9
3.3 Vergleich ausgewählter Arbeiten.....	10
3.4 Forschungslücke und Einordnung der eigenen Arbeit.....	11
4 Anforderungsanalyse.....	12
4.1 Messdaten und Vergleichsbasis.....	12
4.2 Lokale Verarbeitung und Zeitverhalten.....	13
4.3 Evaluation.....	13
4.4 Kernanforderungen und Nachweis.....	14
5 Variantenvergleich und Systementscheidung.....	16
5.1 Bewertungsrahmen und Vorgehen.....	16

5.2 Bewertung der Prüfstandsalternativen.....	17
5.3 Auswahl von Sensorik und Edge-Plattform.....	20
5.4 Datenbasis, Signalrepräsentation und Verfahren.....	22
5.5 Kriterienbasierte Bewertung der Softwarealternativen.....	24
5.6 Systementscheidung und Übergang zur Implementierung.....	27
6 Implementierung.....	29
6.1 Vorhandener Prüfstand und dokumentierte Bedingungen.....	29
6.2 Softwarestruktur und Datenfluss.....	31
6.3 Korrigierte Sensorerfassung und Rohdatenprotokoll.....	33
6.4 Datenaufteilung, Modelle und Schwellen.....	34
6.5 Liveverarbeitung und Ressourceninstrumentierung.....	36
6.6 Technische Prüfbelege und deren Grenzen.....	37
6.7 Kontrollierte Pilotfolge und Vorbereitung der Evaluation.....	39
7 Evaluation und Ergebnisse.....	41
8 Diskussion.....	42
9 Ausblick.....	43
10 Fazit.....	44
11 Literaturverzeichnis.....	i
12 Anhang.....	iv

Abkürzungsverzeichnis

Abkürzung	Bedeutung
AE	Autoencoder
CPU	Central Processing Unit
DFT	Diskrete Fourier-Transformation
FFT	Fast Fourier Transform
FPR	False-Positive-Rate
GPIO	General Purpose Input/Output
GUI	Graphical User Interface (grafische Benutzeroberfläche)
I ² C	Inter-Integrated Circuit
IEPE	Integrated Electronics Piezo-Electric
KI	Künstliche Intelligenz
MCU	Microcontroller Unit
ML	Machine Learning
MSE	Mean Squared Error
ODR	Output Data Rate
PWM	Pulsweitenmodulation
RAM	Random Access Memory
RMS	Root Mean Square
RSS	Resident Set Size
SBC	Single-Board Computer
SPI	Serial Peripheral Interface
TFLite	TensorFlow Lite

1 Einleitung, Problemstellung und Zielsetzung

1.1 Ausgangssituation und Motivation

Beschleunigungssignale ermöglichen es, Schwingungen technischer Systeme zu beobachten. Bei rotierenden Komponenten können Veränderungen dieser Signale auf einen veränderten Zustand hinweisen. Randall und Antoni (2011) erläutern dies am Beispiel der Wälzlagerdiagnose. Die vorliegende Arbeit untersucht die lokale Erkennung solcher Abweichungen anhand von Beschleunigungsdaten.

Die Auswertung soll lokal in der Nähe des Sensors erfolgen. Dieses Prinzip des Edge Computing kann die Übertragung von Rohdaten und die Abhängigkeit von entfernten Rechendiensten verringern (Shi et al., 2016). Neben einer guten Erkennung sind deshalb der Rechen- und Speicherbedarf sowie die Verarbeitungszeit wichtig. Zusätzlich wird untersucht, wie sich veränderte normale Messbedingungen auf die Entscheidungen auswirken.

1.2 Wissenschaftliche Problemstellung

Bei der normalitätsbasierten Anomalieerkennung wird ein Referenzmodell aus normalen Messdaten gebildet. Neue Daten gelten als auffällig, wenn sie ausreichend von dieser Referenz abweichen. Dieses Vorgehen benötigt keine vollständige Sammlung möglicher Fehlerbeispiele. Seine Aussagekraft hängt jedoch davon ab, welche Normalbedingungen die Referenzdaten abdecken (Chandola et al., 2009).

Für einen belastbaren Vergleich müssen die Verfahren unter vergleichbaren Bedingungen bewertet werden. Informationen aus dem Test dürfen weder die Modellwahl noch die Vorverarbeitung oder die Entscheidungsschwelle beeinflussen; andernfalls kann die Erkennungsleistung überschätzt werden (Kaufman et al., 2012). Außerdem muss die gesamte Verarbeitung auf der Zielhardware betrachtet werden. Eine kurze Modellinferenz allein zeigt noch nicht, ob alle eingehenden Daten rechtzeitig verarbeitet werden.

1.3 Zielsetzung der Arbeit

Ziel ist die Konzeption, Umsetzung und experimentelle Bewertung einer lokalen Verarbeitungskette (Pipeline) für mehrkanalige Beschleunigungsdaten. Verglichen werden ein neuronales Hauptverfahren, eine statistische Schwellenwertbaseline und ein klassisches Lernverfahren. Auch das klassische Verfahren gehört zum maschinellen Lernen. Die konkrete Auswahl des Systems und des Versuchsaufbaus wird in Kapitel 5 anhand der Anforderungen und eines Variantenvergleichs begründet.

Der Beitrag ist der gemeinsame Vergleich von Erkennungsqualität und technischem Aufwand. Ein Zusatzversuch untersucht die Grenzen der unveränderten Pipeline bei geänderten Normalbedingungen.

1.4 Forschungsfragen und Hypothesen

Zentrale Forschungsfrage: Wie lässt sich eine lokale Anomalieerkennung für mehrkanalige Beschleunigungsdaten auf ressourcenbeschränkter Edge-Hardware umsetzen und hinsichtlich Erkennungsqualität, Ressourcenbedarf, Zeitverhalten und veränderter Normalbedingungen bewerten?

FF1: Welche Anforderungen ergeben sich aus der Aufgabe, und wie begründen sie die Auswahl von Versuchsaufbau, Sensorik, Hardware und Verfahren?

FF2: Wie unterscheiden sich Hauptverfahren und Baselines bei gleichen Testfenstern und einer einheitlichen Schwellenwertkalibrierung auf Normaldaten hinsichtlich Precision, Recall, F1-Score und Fehlalarmrate?

FF3: Wie viel Speicher und Rechenleistung benötigt die lokale Pipeline, und wie häufig überschreitet ihre Verarbeitung das Zeitbudget bis zum nächsten Fenster?

FF4: Wie verändert sich die Fehlalarmrate bei gezielt geänderten Normalbedingungen, wenn Vorverarbeitung, Modell und Schwelle unverändert bleiben?

Die Arbeitshypothesen legen die Vergleichsgrößen fest. Bereits zur Entwicklung verwendete Daten werden explorativ ausgewertet. Eine unabhängige Bestätigung erfordert neue Testaufnahmen und ein vor deren Auswertung festgelegtes Versuchsprotokoll.

H1 – Erkennungsqualität: Das neuronale Hauptverfahren erreicht bei den definierten Versuchsanomalien einen höheren F1-Score als die statistische Baseline. Diese Erwartung beruht auf der möglichen Nutzung von Informationen, die ein einzelner Kennwert nicht abbildet. Bewertet werden die F1-Differenz auf derselben Testmenge sowie Fehlalarme und übersehene Anomalien.

H2 – Zeitverhalten: Das empirische 99. Perzentil (P_{99}) der Verarbeitungslatenz liegt unter dem Fensterintervall S/f_s . Die Latenz reicht vom vollständigen Fenster bis zur verfügbaren Entscheidung. S bezeichnet die Schrittweite in Messzeitpunkten, f_s die aus Zeitstempeln ermittelte Erfassungsrate. Fristüberschreitungen werden zusätzlich gezählt; P_{99} garantiert keine fristgerechte Ausgabe jeder Entscheidung.

H3 – Veränderte Normalbedingungen: Bei unveränderter Pipeline steigt die Fehlalarmrate unter der untersuchten veränderten Normalbedingung. Verglichen werden die Fehlalarmraten vor und nach der Änderung. Eine Ursache lässt sich nur bei kontrollierten weiteren Einflüssen zuordnen.

1.5 Abgrenzung der Arbeit

Die Ergebnisse gelten für die untersuchten Bedingungen und die im Versuchsprotokoll definierten Anomalien. Deren Erkennung erlaubt weder eine gesicherte Schadensdiagnose noch eine Prognose der Restlebensdauer. Die Übertragung auf andere Systeme und den industriellen Dauerbetrieb erfordert weitere Untersuchungen.

Der Kernversuch bewertet die lokale Inferenz eines zuvor trainierten Modells auf einer Zielplattform. Die Pipeline bleibt dabei unverändert. Eine ergänzende Nachkalibrierung wird gesondert ausgewertet.

1.6 Aufbau der Arbeit

Kapitel 2 und 3 erläutern Grundlagen und Vorarbeiten. Darauf folgen Anforderungen (Kapitel 4), Variantenvergleich und Versuchsaufbau (Kapitel 5), Implementierung (Kapitel 6) und Evaluation (Kapitel 7). Diskussion und Ausblick ordnen Grenzen und weitere Untersuchungen ein; das Fazit beantwortet die Forschungsfragen.

2 Theoretische Grundlagen

Dieses Kapitel erklärt die benötigten Begriffe, mögliche Verfahren und die Messgrößen für die Evaluation. Die dargestellten Ansätze bilden die Grundlage für den Variantenvergleich; ihre Auswahl für das System erfolgt erst in Kapitel 5.

2.1 Zustandsüberwachung und Anomalieerkennung

Zustandsüberwachung (Condition Monitoring) erfasst und bewertet den technischen Zustand. Diagnose ordnet Fehlerursachen zu, Prognose schätzt die künftige Zustandsentwicklung (Jardine et al., 2006). Anomalieerkennung bewertet zunächst nur Abweichungen von einer Normalreferenz.

Was als anomal gilt, hängt von Referenz und Betriebskontext ab. Auch ein zeitlicher Verlauf kann auffällig sein, obwohl seine Einzelwerte unauffällig sind (Chandola et al., 2009). Deshalb werden hier Messfenster bewertet.

Klassische und neuronale Verfahren können auf als normal ausgewählten Daten angepasst werden. Dieses Normaldaten-Szenario beschreibt die Trainingsgrundlage und ist von der Algorithmusklasse zu unterscheiden. Der Anomalie-Score ist eine Bewertungsgröße, zunächst keine Schadenswahrscheinlichkeit (Ruff et al., 2021).

2.2 Verfahrensklassen für die Anomalieentscheidung

2.2.1 Statistische Schwellenwertverfahren

Der Effektivwert (Root Mean Square, RMS) eines Kanals mit N Messwerten ist die Quadratwurzel des Mittelwerts der quadrierten Werte:

(2-1)

Der RMS fasst die Amplitude eines Fensters zusammen, erhält aber die Reihenfolge der Werte nicht. Für den Fenstermittelwert $\bar{x}$ gilt $\text{RMS}^2(x) = \bar{x}^2 + \text{RMS}^2(x - \bar{x})$. Ohne Mittelwertentfernung beeinflussen daher auch konstante Anteile den Kennwert.

Beschleunigungssensoren erfassen neben Bewegung unter anderem die Gravitation und einen Sensoroffset (Analog Devices, 2022). Rohwert-RMS und RMS nach

Mittelwertentfernung sind deshalb zu unterscheiden. Ebenso muss feststehen, wie mehrere Messkanäle zusammengefasst werden.

2.2.2 Klassische Verfahren im Normaldaten-Szenario

Isolation Forest nutzt zufällige Teilungsbäume. Jeder Schritt teilt die Daten anhand eines zufällig gewählten Merkmals und Teilungspunkts. Beobachtungen, die im Mittel durch wenige Teilungen isoliert werden, gelten als auffälliger (Liu et al., 2008). Im Normaldaten-Szenario erfolgt seine Anpassung ausschließlich anhand normaler Beobachtungen.

Die Eingabe ist ein Vektor fester Länge aus Fensterkennwerten oder aneinandergereihten Messwerten. Die verwendete Darstellung und die Richtung des ausgegebenen Scores müssen dokumentiert werden.

2.2.3 Autoencoder

Ein Autoencoder erzeugt aus einer Eingabe x eine Rekonstruktion $\hat{x}$. Der Encoder bildet die Eingabe auf eine interne Darstellung ab, der Decoder rekonstruiert daraus die Ausgangswerte. Eine begrenzte Modellkapazität oder Regularisierung soll bloßes Kopieren verhindern (Goodfellow et al., 2016). Als Anomalie-Score kann der mittlere quadratische Rekonstruktionsfehler verwendet werden:

(2-2)

Dabei bezeichnet d die Zahl der verglichenen Einzelwerte. Für ein Fenster mit H Messzeitpunkten und C Kanälen gilt $d = HC$. Der Score hängt von der Skalierung und der Mittelung ab. Bei standardisierten Eingaben ist er dimensionslos.

Die Annahme lautet, dass ein auf Normaldaten trainierter Autoencoder auffällige Eingaben schlechter rekonstruiert. Das ist keine allgemeine Garantie: Auch bestimmte Anomalien können gut rekonstruiert werden (Ruff et al., 2021). Ob der Score die definierten Versuchsanomalien trennt, muss daher experimentell geprüft werden.

2.3 Messdaten und Fensterbildung

Bei dreiachsigen Messungen enthält jeder Messvektor beispielsweise einen X-, Y- und Z-Wert. Sensorausgaberate, Signalbandbreite und Abfragerate der Software sind unterschiedliche Größen. Häufigeres Auslesen erzeugt nicht automatisch neue Sensorwerte.

Die Abstimmung von Sensorkonfiguration und Datenabruf ist daher zu prüfen (Analog Devices, 2022).

Ein Fenster bündelt H aufeinanderfolgende Messvektoren. Bei C Messkanälen enthält es HC Werte. Die Schrittweite S bestimmt, nach wie vielen neuen Messzeitpunkten das nächste Fenster beginnt. Bei gleichmäßiger Erfassung beträgt das Fensterintervall S/f_s . Für die Zeitbewertung wird f_s aus den Zeitstempeln bestimmt; ob jeder erfasste Vektor eine neue Sensormessung enthält, wird getrennt geprüft.

Bei ungleichmäßiger Erfassung und Datenlücken reichen nominelle Zeitangaben nicht aus. Die tatsächlich erfassten Zeitstempel müssen deshalb erhalten bleiben.

2.4 Datenaufteilung und Vorverarbeitung

Trainingsdaten dienen zur Anpassung des Modells. Validierungsdaten unterstützen die Auswahl von Einstellungen und die Kalibrierung. Die Testdaten bleiben bis zur abschließenden Bewertung ungenutzt. Vollständige Aufnahmen bilden dabei Gruppen und werden vor der Fensterbildung getrennt zugeordnet. So werden ähnliche Abschnitte derselben Aufnahme nicht auf Training und Test verteilt (scikit-learn developers, o. J. b).

Data Leakage liegt vor, wenn Informationen in die Entwicklung einfließen, die bei der vorgesehenen Anwendung nicht verfügbar wären. Ein Beispiel ist die Standardisierung mit Mittelwert und Standardabweichung der gesamten Datenmenge. Solche Parameter dürfen nur aus Trainingsdaten bestimmt werden. Danach werden sie unverändert auf Validierungs-, Test- und Live-Daten angewendet (scikit-learn developers, o. J. a).

Fenster derselben Aufnahme sind keine unabhängigen Messläufe. Auch getrennte Aufnahmen können bei gleicher Montage sehr ähnlich sein. Die Aussagekraft hängt daher von Zahl und Vielfalt der Messläufe ab.

2.5 Schwellenwertkalibrierung

Aus dem Anomalie-Score wird durch einen Schwellenwert eine Entscheidung. Für den Vergleich werden die Scores so ausgerichtet, dass größere Werte eine stärkere Auffälligkeit bedeuten. Die Entscheidung lautet dann *anomal*, wenn der Score die festgelegte Schwelle überschreitet.

Eine mögliche Kalibrierung nutzt ein hohes empirisches Quantil der Scores separater Normaldaten. Ein Quantil bezeichnet einen Grenzwert in der sortierten Verteilung. Die konkrete Quantilsregel wird für alle Verfahren vor der Testauswertung festgelegt. Damit wird dieselbe Kalibrierungsregel verwendet, auch wenn die numerischen Schwellen verschieden sind.

Eine so bestimmte Schwelle garantiert keine bestimmte Fehlalarmrate auf späteren Aufnahmen. Dafür müssen die neuen Normaldaten ausreichend zur Kalibrierungsgrundlage passen. Eine nachträgliche Anpassung anhand der Testgüte würde die unabhängige Bewertung aufheben (scikit-learn developers, o. J. a).

2.6 Erkennungsqualität und Aussagekraft der Evaluation

Die positive Klasse ist die definierte Anomalie. Die Konfusionsmatrix zählt richtig erkannte Anomalien (TP), Fehlalarme (FP), übersehene Anomalien (FN) und richtig erkannte Normalfenster (TN). Daraus ergeben sich die Kennzahlen in Tabelle 2-1 (scikit-learn developers, o. J. c).

Tabelle 2-1: Kennzahlen der binären Anomalieerkennung

Kennzahl	Berechnung	Bedeutung
Precision	$TP / (TP + FP)$	Anteil richtiger Anomalien unter allen Alarmen
Recall	$TP / (TP + FN)$	Anteil erkannter Anomalien unter allen tatsächlichen Anomalien
F1-Score	$2TP / (2TP + FP + FN)$	Harmonisches Mittel von Precision und Recall
Fehlalarmrate (FPR)	$FP / (FP + TN)$	Anteil falsch gemeldeter Fenster unter allen Normalfenstern
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Anteil aller richtigen Entscheidungen

Eine hohe Accuracy kann bei ungleichen Klassenanteilen täuschen. Deshalb werden die Konfusionsmatrix und mehrere Kennzahlen gemeinsam berichtet. Nicht definierte Werte, etwa Recall ohne Anomalien im Datensatz, werden gekennzeichnet. Für reine Normalaufnahmen steht die Fehlalarmrate im Mittelpunkt. Zusätzlich werden Klassenanteile und Ergebnisse je Aufnahme angegeben.

2.7 Lokale Ausführung und Zeitverhalten

Bei lokaler Ausführung wird das trainierte Modell nahe der Datenquelle ausgewertet (Shi et al., 2016). Ob Rechenleistung und Arbeitsspeicher der Zielplattform ausreichen, muss für die konkrete Anwendung gemessen werden.

Die Modellgröße bezeichnet die gespeicherte Datei. Der Prozessspeicher umfasst zusätzlich unter anderem Bibliotheken, Eingabedaten und Zwischenergebnisse. Modellgröße und Speicherbedarf während der Ausführung sind daher getrennt auszuweisen. Auch die CPU-Auslastung benötigt eine dokumentierte Messdauer und Bezugsgröße.

Die Inferenzzeit umfasst nur die Modellauswertung. Die Verarbeitungslatenz reicht in dieser Arbeit vom vollständigen Fenster bis zur verfügbaren Entscheidung und enthält auch Vorverarbeitung und Wartezeiten. Die Zeit zum Sammeln des Fensters ist darin nicht enthalten. Ein Echtzeitsystem muss seine Ergebnisse innerhalb festgelegter Fristen liefern (Buttazzo, 2011). Mittelwerte allein belegen das nicht.

P95 und P99 beschreiben hohe empirische Perzentile der gemessenen Latenzen. Sie werden zusammen mit Fristüberschreitungen und Erfassungspausen ausgewertet. Trainierte Modelle können für die lokale Ausführung in ein anderes Format konvertiert werden (Google, 2026a). Danach müssen Scores und Entscheidungen erneut geprüft werden.

2.8 Veränderte Normalbedingungen

Ändern sich Montage, Sensororientierung oder Betriebspunkt, können sich normale Eingabedaten von der bisherigen Referenz unterscheiden. Solche Verteilungsänderungen werden häufig als Domain Shift bezeichnet. Sie können die Anomalieerkennung beeinträchtigen (Ruff et al., 2021).

FF4 untersucht deshalb die Fehlalarmrate bei unveränderter Pipeline unter geänderten Normalbedingungen. Eine anschließende Nachkalibrierung ist als eigener Versuch auszuwerten.

3 Stand der Technik und verwandte Arbeiten

Dieses Kapitel vergleicht ausgewählte Originalstudien nach Datenbasis, Lernaufgabe, lokaler Ausführung und Übertragbarkeit. Die gezielte Auswahl mit Recherchestand September 2026 umfasst Arbeiten zu den Forschungsfragen; sie ist keine vollständige systematische Literaturübersicht.

3.1 Datenbasis und Messbedingungen

Lessmeier et al. (2016) untersuchen die überwachte Unterscheidung gesunder Lager sowie von Innen- und Außenringschäden. Sie vergleichen künstlich erzeugte Schäden mit Schäden aus beschleunigten Lebensdauerversuchen. Die Klassifikationsgüte hängt dabei von Schadensherkunft und Verfahren ab. Der Befund betrifft überwachte Lagerklassifikation und lässt sich nicht unmittelbar auf normaldatenbasiertes Training übertragen.

Pittino et al. (2020) untersuchen eine Waferbearbeitungsmaschine bei unterschiedlichen Drehzahlen und Verschleißzuständen. Normaldaten stammen aus funktionsfähigen Kammern, Fehlerdaten aus einem Demonstrator ohne Schmierung. Verglichen werden regressionsbasierte Regelkarten und eine überwachte Support-Vector-Machine-Klassifikation. Letztere nutzt zusätzlich Fehlerbeispiele. Ihr Ergebnis belegt deshalb keinen allgemeinen Vorteil im Normaldaten-Szenario.

Knap et al. (2026) vergleichen klassische und tiefe Klassifikationsverfahren in sechs Übertragungsszenarien auf CWRU- und Paderborner Lagerdaten. Ihr beschriebenes Protokoll trennt vollständige Aufnahmen vor der Fensterbildung. Für die eigene Untersuchung folgt daraus die Trennung ganzer Messläufe. Geänderte Normalbedingungen werden zusätzlich in einem eigenen Versuch betrachtet.

3.2 Verfahren und Edge-Ausführung

Ren et al. (2021) führen mit TinyOL einen Autoencoder auf einem Cortex-M4-Mikrocontroller aus und ermöglichen das lokale Nachlernen seiner letzten Schicht. Dreiachsige Lüftermessungen werden vor der Modelleingabe durch Hauptkomponentenanalyse auf eine Dimension reduziert. Nach erneuter Anbringung des Sensors verändert sich der Rekonstruktionsfehler; die feste Schwelle verursacht mehr

Fehlalarme. Die untersuchte Modellanpassung ist von der Bewertung einer unveränderten Pipeline zu unterscheiden.

Antonini et al. (2023) entwickeln einen ESP32-Sensorknoten für die Pumpenüberwachung. Merkmalsbildung, Isolation-Forest-Training und Inferenz laufen lokal; das Training setzt normalen Betrieb voraus. Die Ressourcenmessungen erfolgen am Prüfstand und unterscheiden Laden, Training und Inferenz. Eine Bewertung der Erkennungsgenauigkeit ist ausdrücklich nicht Gegenstand der Studie.

Garay et al. (2026) vergleichen vier Algorithmen in fünf Varianten, darunter Isolation Forest und einen Autoencoder, auf einem Cortex-M4F. Nach einer Sensorneumontage treten mehr Fehlalarme auf; gleichzeitig verändert sich die Umgebungstemperatur. Ein isolierter Montageeffekt ist daraus nicht ableitbar. Die Autoren untersuchen außerdem eine Nachkalibrierung von Skalierung und Schwelle. Beim Isolation Forest stammen die Erkennungskennzahlen vom Modell mit 100 Bäumen, die Mikrocontrollerwerte von einer Variante mit sechs Bäumen. Beide Größen beschreiben somit unterschiedliche Modellstände.

3.3 Vergleich ausgewählter Arbeiten

Tabelle 3-1 fasst die drei Studien mit eigener Hardwareausführung zusammen. Wegen unterschiedlicher Daten und Messgrenzen erlauben ihre Kennzahlen keine unmittelbare Rangfolge.

Tabelle 3-1: Ausgewählte Vorarbeiten und ihre Bedeutung für die Untersuchung

Arbeit	Daten und Lernaufgabe	Ausführung / Messung	Einordnung für diese Arbeit
Ren et al. (2021)	Lüfter; Autoencoder und Nachlernen	Cortex-M4; Inferenz und Online-Anpassung	Belegt Montageempfindlichkeit; Anpassung getrennt von fester Normalreferenz bewerten
Antonini et al. (2023)	Lüfterprüfstand; Isolation Forest mit Normaltraining	ESP32; Laden, Training und Inferenz	Ressourcen getrennt erfassen; keine Erkennungsbewertung in der Studie
Garay et al. (2026)	Unwuchtversuch; fünf normaltrainierte Modellvarianten	Cortex-M4F; Montage- und Temperaturänderung	Modellstände abgleichen; Nachkalibrierung getrennt bewerten

3.4 Forschungslücke und Einordnung der eigenen Arbeit

Die Vorarbeiten zeigen, dass lokale Anomalieerkennung und die Anpassung an veränderte Messbedingungen bereits untersucht werden. Für die eigene Untersuchung bleibt zu klären, welchen zusätzlichen Erkennungsnutzen ein neuronales Verfahren gegenüber den beiden Baselines bietet und welchen technischen Aufwand es verursacht. Die Arbeit liefert hierzu eine anwendungsbezogene Vergleichsevaluation.

Dafür werden drei Aspekte verbunden: gleiche Testfenster und eine gemeinsame Kalibrierungsregel für FF2, Messungen am tatsächlich ausgeführten Modell für FF3 sowie ein kontrollierter Vergleich unveränderter Pipelines bei geänderten Normalbedingungen für FF4. Damit werden die in den Vorarbeiten erkennbaren Grenzen von Vergleichbarkeit und Übertragbarkeit gezielt berücksichtigt.

Kapitel 4 übersetzt diesen Untersuchungsbedarf in überprüfbare Anforderungen und beantwortet damit den ersten Teil von FF1. Kapitel 5 vergleicht anschließend geeignete Lösungsvarianten und begründet die Auswahl des Systems und des Versuchsaufbaus.

4 Anforderungsanalyse

Dieses Kapitel leitet aus den Forschungsfragen, den Grundlagen und den Grenzen der Vorarbeiten überprüfbare Anforderungen ab. Sie beschreiben die benötigten Funktionen und Bewertungskriterien unabhängig von einer konkreten technischen Lösung.

4.1 Messdaten und Vergleichsbasis

Für die Untersuchung muss ein reproduzierbarer Testaufbau realisiert werden (A11). Er muss Messungen im Normalzustand und unter gezielt hergestellten Anomaliebedingungen ermöglichen. Eine Normalbedingung muss für FF4 unabhängig von der Anomaliedefinition veränderbar sein (A10). Ein sicherer Betrieb ist Voraussetzung für die Messläufe.

Versuchsbedingungen, Rücksetzbarkeit und Ablauf sind so zu dokumentieren, dass Messläufe wiederholt werden können.

Die Messdaten müssen die benötigten Beschleunigungskanäle mit Einheit, Zeitstempeln und Aufnahmekennung enthalten (A1). Messbereich, nutzbare Bandbreite, Signal-Rausch-Abstand und zeitliche Erfassung sind für die Versuchszustände zu prüfen. Akzeptanzgrenzen werden vor der abschließenden Testserie festgelegt. Datenlücken, ungültige Werte und Sättigung sind zu kennzeichnen. Für Normal- und Anomaliedaten muss feststehen, ob die Kennzeichnung ganze Aufnahmen oder einzelne Zeitabschnitte beschreibt.

Vollständige Aufnahmen werden vor der Fensterbildung getrennt Training, Validierung oder Test zugeordnet. Fenster dürfen keine Aufnahmegrenzen überschreiten. Training und Schwellenwertkalibrierung verwenden nur Normaldaten. Die Vorverarbeitung wird nach den Regeln aus Abschnitt 2.4 angepasst und unverändert auf spätere Daten angewendet.

Das neuronale Hauptverfahren und die beiden Baselines müssen dieselben Testfenster und Labels verwenden. Fensterlänge und Schrittweite werden gemeinsam festgelegt.

Unterschiedliche Eingabedarstellungen und der Umgang mit unvollständigen Fenstern sind zu dokumentieren. Die Schwellen folgen derselben Kalibrierungsregel auf separaten Normaldaten. Modellwahl und Kalibrierung müssen vor der Testauswertung abgeschlossen sein.

4.2 Lokale Verarbeitung und Zeitverhalten

Datenerfassung, Vorverarbeitung und Entscheidung müssen nach dem Start automatisch und ohne Cloud-Dienst auf der Zielhardware laufen (A4). Die Softwareumgebung muss die benötigten Modelloperationen und Schnittstellen unterstützen; Abhängigkeiten und Versionen sind zu dokumentieren (A5, A6). Für jedes gültige Fenster sind Score, Schwelle und die Entscheidung normal/anomal zu speichern. Ungültige Fenster dürfen nicht als normal bewertet werden. Der Betrieb muss ohne Speicherabbruch und ohne stetig wachsenden Verarbeitungsrückstand möglich sein (A8).

Messläufe müssen gezielt gestartet und geordnet beendet werden können (A13). Jeder Lauf erhält eine eindeutige Kennung. Beim Beenden müssen die bis dahin vollständig erfassten Daten und Ergebnisse gespeichert bleiben. Messdaten, Entscheidungen und die zugehörigen Angaben müssen maschinenlesbar vorliegen und eindeutig zusammengehören (A5).

Als optionale Erweiterung kann eine grafische Benutzeroberfläche zur Live-Anzeige (Live-GUI) entwickelt werden (A12). Sie zeigt den aktuellen Erfassungsstatus sowie Score, Schwelle, Entscheidung und Zeitstempel des zuletzt ausgewerteten Fensters. Fehlende oder veraltete Daten müssen erkennbar sein. Die Anzeige verwendet die Ergebnisse der Verarbeitung und verändert weder Modell noch Schwelle. Der Messbetrieb und die Evaluation müssen auch ohne diese Oberfläche möglich sein.

Für FF3 werden Inferenzzeit und Verarbeitungslatenz nach Abschnitt 2.7 getrennt gemessen. Das Ziel aus H2 ist $P99 \text{ der Verarbeitungslatenz} < S/f_s$. Zusätzlich sind Anzahl und Anteil der Fristüberschreitungen sowie Erfassungspausen zwischen Fenstern zu berichten.

4.3 Evaluation

Für FF2 werden die Kennzahlen aus Abschnitt 2.6 auf derselben Testmenge ermittelt. Anzahl der Aufnahmen, Klassenverteilung und Ergebnisse je Aufnahme sind anzugeben (A9). Für FF3 werden Modellgröße, CPU-Auslastung und Prozessspeicher einschließlich Bibliotheken und Vorverarbeitung auf derselben Zielhardware gemessen (A8). Messbedingungen sowie Software-, Code- und Modellstand müssen nachvollziehbar sein (A5). Bei einer Modellkonvertierung sind numerische Abweichungen und Entscheidungen gegenüber dem

Ausgangsmodell zu prüfen; die Prüfdaten und Akzeptanzgrenzen werden vor der Testauswertung festgelegt (A6). Wird die Live-GUI umgesetzt, ist ihr zusätzlicher Einfluss auf Latenz und Ressourcenbedarf bei ansonsten gleichen Bedingungen mit und ohne Anzeige zu messen.

Für FF4 wird eine dokumentierte Normalbedingung gezielt verändert. Vorverarbeitung, Modell und Schwelle bleiben dabei unverändert. Verglichen werden die Fehlalarmraten unter der ursprünglichen und der veränderten Bedingung. Eine Nachkalibrierung ist getrennt auszuwerten.

4.4 Kernanforderungen und Nachweis

Tabelle 4-1 enthält die Anforderungen A1–A13 mit Art, Priorität und Nachweis. F kennzeichnet funktionale Anforderungen an Systemfunktionen. NF bezeichnet nichtfunktionale Anforderungen an Nachvollziehbarkeit, Zeitverhalten und Betriebsstabilität. Methodische Anforderungen (M) betreffen Versuchsplanung und wissenschaftliche Auswertung. MUSS ist verbindlich; KANN bezeichnet die optionale Live-GUI. Ihr Nachweis ist nur bei Umsetzung erforderlich. Kapitel 6 dokumentiert Umsetzung und Prüfprotokoll. Kapitel 7 bewertet die Erfüllung und prüft die Hypothesen. Nichterfüllte technische Ziele und nicht bestätigte Hypothesen werden als Ergebnisse offengelegt.

Tabelle 4-1: Anforderungen, Prioritäten und Nachweise

ID	Art / Priorität	Anforderung	Nachweis
A1	F MUSS	Messdaten mit überprüfter Qualität erfassen	Kanäle, Einheiten und Zeitstempel prüfen; Bandbreite, Signal-Rausch-Abstand, Sättigung und Datenlücken anhand vorab festgelegter Grenzen bewerten
A2	M MUSS	Datenbasis nach Aufnahmen trennen	Ganze Aufnahmen zuordnen; Training und Kalibrierung nur mit Normaldaten; Vorverarbeitung nur an Trainingsdaten anpassen
A3	M MUSS	Verfahren einheitlich vergleichen	Gleiche Testfenster und Labels; gemeinsame Kalibrierungsregel und dokumentierte Eingaben
A4	F MUSS	Automatisch und lokal auswerten	Versuchslauf ohne Cloud-Dienst; Score, Schwelle und Entscheidung je gültigem Fenster; ungültige Fenster kennzeichnen

ID	Art / Priorität	Anforderung	Nachweis
A5	NF MUSS	Daten und Softwarestände nachvollziehbar zuordnen	Lauf, Fenster, Code, Modell, Vorverarbeitung sowie Softwareabhängigkeiten und Versionen maschinenlesbar dokumentieren
A6	M MUSS	Zielkompatibilität und ausgeführtes Modell prüfen	Modelloperationen auf der Zielhardware prüfen; bei Konvertierung Scores und Entscheidungen nach vorab festgelegten Akzeptanzgrenzen vergleichen
A7	NF MUSS	Latenzziel aus H2 erfüllen: $P99 < S/f_s$	Inferenzzeit und Verarbeitungslatenz getrennt erfassen; Latenzverteilung, Fristüberschreitungen und Erfassungspausen berichten
A8	NF MUSS	Ohne Speicherabbruch und wachsenden Rückstand laufen	Modellgröße, Prozessspeicher, CPU und Rückstand über dokumentierte Messdauer erfassen; Aufwärmphase und Hintergrundlast angeben
A9	M MUSS	Erkennungsqualität bewerten	Konfusionsmatrix, Precision, Recall, F1, Fehlalarmrate und Accuracy auf derselben Testmenge; Ergebnisse je Aufnahme
A10	M MUSS	Geänderte Normalbedingungen prüfen	Fehlalarmraten bei unveränderter Vorverarbeitung, unverändertem Modell und unveränderter Schwelle vergleichen
A11	M MUSS	Reproduzierbaren Testaufbau realisieren	Aufbau und Protokoll dokumentieren; sicheren Betrieb herstellen; Normal- und Anomaliezustände wiederholen; Rücksetzbarkeit und kontrollierte Veränderung prüfen
A12	F KANN	Live-GUI zur Ergebnisanzeige entwickeln	Angezeigte Werte mit gespeicherten Ergebnissen abgleichen; veraltete und fehlende Daten prüfen; Zusatzaufwand messen
A13	F MUSS	Messläufe starten und geordnet beenden	Start und Stopp prüfen; eindeutige Laufkennung und Erhalt vollständig erfasster Daten und Ergebnisse nachweisen

Kapitel 5 leitet aus A1–A13 Kriterien für Prüfstand, Messkette, Verarbeitung und Evaluation ab. MUSS-Prioritäten sind keine Gewichte: Bekannte Nichterfüllungen dürfen nicht durch Vorteile anderer Kriterien ausgeglichen werden. Noch offene Nachweise begrenzen die Vorauswahl. Die konkrete Lösung wird erst im Variantenvergleich festgelegt.

5 Variantenvergleich und Systementscheidung

Dieses Kapitel begründet die Systemauswahl anhand der Anforderungen A1–A13 und beantwortet damit den zweiten Teil von FF1. Zunächst wird der Prüfstand ausgewählt. Darauf folgen Sensorik und Edge-Plattform, Datenrepräsentation und Verfahren sowie die Entwicklungs- und Inferenzumgebung. Die abschließende Systementscheidung verbindet diese Teilauswahlen mit den noch ausstehenden Nachweisen.

5.1 Bewertungsrahmen und Vorgehen

Die Bewertung berücksichtigt die Eignung für die Forschungsfragen, die technische Umsetzbarkeit und den erforderlichen Zusatzaufwand. Anforderungen und Auswahlgründe müssen dabei nachvollziehbar zusammenhängen (NASA, 2023). Für den Prüfstand stehen wiederholbare Zustände und kontrollierte Veränderungen im Vordergrund (A10, A11). Für die Messkette sind Signalqualität und zeitliche Erfassung entscheidend (A1). Hardware und Software müssen die lokale Verarbeitung sowie ihre überprüfbare Ausführung ermöglichen (A4–A8).

Zuerst werden die MUSS-Anforderungen geprüft. Eine bekannte Nichterfüllung kann nicht durch Punkte in anderen Kriterien ausgeglichen werden. Noch offene Nachweise, etwa zur Signalqualität oder Latenz, führen zu einer vorläufigen Auswahl mit Prüfvorbehalt.

Sicherheitsmaßnahmen sind vor dem Versuch umzusetzen. Erst unter diesen Voraussetzungen werden geeignete Entwurfsvarianten gegeneinander abgewogen.

Für Prüfstand, Edge-Plattform und Software werden gewichtete Nutzwertmatrizen verwendet. Die Punkteskala reicht von 0 bis 4; ein höherer Wert bedeutet einen höheren Nutzen für dieses Projekt. Die Kriterientabellen erläutern die Bewertungsmaßstäbe.

Zwischenwerte 1 und 3 kennzeichnen abgestufte Einschätzungen zwischen den beschriebenen Ankern. Gleich große Punktabstände werden vereinfachend als gleich große Nutzenzuwächse behandelt. Das ist eine Modellannahme und keine gemessene Leistungsrelation.

Die Gewichte beziehen sich auf den Unterschied zwischen dem ungünstigsten und dem günstigsten beschriebenen Zustand eines Kriteriums. Je größer dessen Bedeutung für die Forschungsfragen, desto höher wird dieser Unterschied gewichtet. Dieses Vorgehen

orientiert sich am Prinzip der Swing-Gewichtung (Department for Communities and Local Government, 2009, S. 63–65). Die konkreten Gewichte und Punkte sind eigene Entwurfsannahmen; sie stammen weder aus Herstellerangaben noch aus einer Expertenbefragung. Jede Matrix wird für sich auf 100 % normiert. Nutzwerte verschiedener Auswahlbereiche sind nicht miteinander vergleichbar.

Dabei ist m die Anzahl der Kriterien, N_j der Nutzwert der Alternative j , w_i das in Prozent angegebene Gewicht des Kriteriums i und s_{ij} dessen Punktwert. Der Nutzwert liegt zwischen 0 und 4. Er ist weder eine Erfolgswahrscheinlichkeit noch ein Nachweis erfüllter Anforderungen.

Die Sensitivität wird rechnerisch geprüft: Jedes Gewicht wird einzeln um fünf Prozentpunkte erhöht und gesenkt; die übrigen Gewichte werden proportional auf zusammen 100 % angepasst. Zusätzlich werden gleiche Gewichte und ausgewählte Änderungen unsicherer Punktwerte um einen Punkt betrachtet. Diese Prüfungen zeigen die Abhängigkeit von den Annahmen, ersetzen aber keine Messung (NASA, 2023).

5.2 Bewertung der Prüfstandsalternativen

Der Prüfstand muss normale und gezielt veränderte Zustände wiederholt herstellen können (A11). Für FF4 muss außerdem eine Normalbedingung veränderbar sein, ohne diese Änderung bereits als Anomalie zu kennzeichnen (A10). Tabelle 5-1 vergleicht hierfür drei Alternativen anhand ihrer Kontrollierbarkeit, des zusätzlichen Aufwands und ihrer Aussagegrenzen.

Tabelle 5-1: Bewertung der Prüfstandsalternativen

Alternative	Kontrollierbarkeit und Wiederholung	Zusatzaufwand und Aussagegrenze
Lüfter mit PWM-Ansteuerung	Betriebspunkte vorgebar; Versuchszustände bei fester Montage wiederholbar zu gestalten	Wenige Antriebskomponenten; Lastvariation und Übertragbarkeit auf Industriemaschinen begrenzt
Servomotor mit Regler	Geregelte Drehzahl- und Bewegungsprofile; Lastvariation mit zusätzlicher Belastungseinrichtung	Regler, mechanische Kopplung und gegebenenfalls Lastmaschine nötig; Regelung beeinflusst das Schwingungsverhalten
Motor-/ Lagerprüfstand	Drehzahl, Last und Lagerzustände gezielt untersuchbar	Zusätzliche Mechanik, Messtechnik und Schutzmaßnahmen; größere Nähe zur Lagerdiagnostik

Quelle: Eigene Bewertung auf Grundlage von ARCTIC (o. J.), Oriental Motor (o. J.) und Universität Paderborn (o. J.). Der Aufwand ist eine qualitative Einschätzung der benötigten Komponenten.

Für die Bewertung wird vorausgesetzt, dass bei allen drei Varianten sichere, rücksetzbare Versuchszustände realisiert werden können. Dies bleibt nach A10 und A11 am fertigen Aufbau nachzuweisen. Die folgenden Kriterien unterscheiden die verbleibenden Gestaltungsmöglichkeiten. Der Realisierungsaufwand erhält das größte Gewicht, weil die Arbeit den Methodenvergleich und keine Entwicklung einer komplexen Antriebsanlage untersucht. Es werden benötigte Baugruppen eingeschätzt; Kosten oder Arbeitsstunden werden nicht behauptet.

Tabelle 5-2: Kriterien und Gewichte für den Prüfstand

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
P1 Betriebspunkte einstellen	25 %	A10, A11. 0: keine Vorgabe; 2: einzelne Stellgröße ohne unabhängige Lastregelung; 4: geregelte Drehzahl und zusätzlich Last oder Bewegung vorgebar.
P2 Einflüsse trennen	25 %	A10, A11. 0: Änderungen nicht zuordenbar; 2: mehrere gekoppelte Einflüsse; 4: einfache Variation bei fester Mechanik und wenigen Einflussgrößen.
P3 Aufbauaufwand	40 %	A11. 0: im Projekt nicht realisierbar; 2: Antrieb, Regler und Kopplung aufzubauen; 4: integrierter Antrieb mit wenigen Zusatzbaugruppen.
P4 Anwendungsnähe	10 %	FF1, FF4. 0: kein rotierendes System; 2: einfache rotierende Baugruppe; 4: gezielt untersuchbare Maschinen- und Lagerzustände.

Quelle: Eigene Kriterien und Gewichte, abgeleitet aus Kapitel 4. Maßstäbe beschreiben projektbezogene Präferenzen.

Tabelle 5-3: Gewichtete Bewertung der Prüfstandsalternativen

Alternative	P1	P2	P3	P4	Nutzwert
Lüfter mit PWM	2	4	4	2	3,30
Servomotor mit Regler	4	3	2	3	2,85
Motor-/Lagerprüfstand	4	3	1	4	2,55

Quelle: Eigene Punktvergabe auf Grundlage des Variantenvergleichs und der vorausgehenden Kriterien; keine experimentellen Messwerte.

Der Lüfter erhält weniger Punkte bei der Betriebspunktregelung und der Nähe zur Maschinenüberwachung, bietet aber einen einfachen Aufbau mit wenigen zu kontrollierenden Einflussgrößen. Beim Servo kommen Rückführung und mechanische Kopplung hinzu. Der Motor-/Lagerprüfstand ermöglicht die gezielteste Lageruntersuchung, benötigt jedoch zusätzliche Belastungs- und Messbaugruppen. Die Punkte bewerten diese angenommenen Aufbauvarianten; sie belegen keine schlechtere Reproduzierbarkeit eines professionellen Prüfstands.

Unter den gewählten Gewichten wird der Lüfter mit 3,30 Punkten bevorzugt. Beispielhaft ergibt sich $0,25 \cdot 2 + 0,25 \cdot 4 + 0,40 \cdot 4 + 0,10 \cdot 2 = 3,30$. In den acht einzelnen Gewichtsänderungen bleibt er vorn. Bei gleichen Gewichten erreichen dagegen alle Varianten 3,00 Punkte. Die Auswahl hängt damit wesentlich vom begrenzten Realisierungsaufwand ab. Eine einzelne Änderung eines Punktwerts um einen Punkt kehrt die Ausgangsrangfolge an der Spitze nicht um; gemeinsame Änderungen können dies tun. Für den Kernversuch wird deshalb der Lüfter gewählt. Ein Servomotor wäre bei Lastwechseln oder dynamischen Bewegungsprofilen sinnvoll, ein Motor-/Lagerprüfstand bei konkreten Lagerfehlern. Diese Aufgaben gehören nicht zum Kernvergleich. Die Ergebnisse bleiben auf den untersuchten Laborversuch begrenzt.

Als konkrete Variante wird der ARCTIC P12 Pro PST vorgesehen, dessen Datenblatt PWM-Ansteuerung und ein Drehzahlsignal ausweist (ARCTIC, o. J.). Ein PWM-Tastgrad ist eine Stellgröße und kein Nachweis einer konstanten Drehzahl; die tatsächliche Drehzahl ist zu erfassen. Für FF4 wird ein zweiter normaler Betriebspunkt bei unveränderter Montage untersucht. Art und Rücksetzbarkeit der Versuchsanomalien, Schutzmaßnahmen und Messablauf werden in Kapitel 6 vor der abschließenden Testserie festgelegt.

5.3 Auswahl von Sensorik und Edge-Plattform

Die Sensorik muss die vorgesehenen dreiachsigen Beschleunigungssignale mit ausreichender Qualität erfassen. Verglichen werden zwei digital anschließbare MEMS-Sensoren und eine industrielle Messkette. Tabelle 5-4 nennt Eigenschaften, die für A1 relevant sind.

Datenblattwerte belegen noch keine ausreichende Signalqualität am späteren Montageort.

Tabelle 5-4: Vergleich der Sensorik und Messketten

Alternative	Relevante Herstellerangaben	Bewertung für den Prüfstand
ADXL345	Drei Achsen; $\pm 2/4/8/16$ g; I ² C/SPI; FIFO	Direkte digitale Erfassung; Bandbreite, Auflösung und Rauschen am Aufbau prüfen
ADXL355	Drei Achsen; $\pm 2/4/8$ g; 20-Bit-ADC; typische Rauschdichte $22,5 \mu\text{g}/\sqrt{\text{Hz}}$ bei ± 2 g	Alternative bei höheren Anforderungen an Auflösung und Rauschen; eigene Integrationsprüfung nötig
PCB 356A32 mit IEPE-Messkette	Drei Achsen; ± 50 g; 100 mV/g ; 1–4000 Hz innerhalb $\pm 5 \%$	Größerer spezifizierter Frequenzbereich; Konstantstromspeisung und externe A/D-Wandlung nötig

Quelle: Eigene Einordnung nach Analog Devices (2022, 2025) und PCB Piezotronics (o. J.). Die Zahlen sind Herstellerangaben.

Für den ersten Systementwurf wird der ADXL345 mit digitaler I²C-Anbindung vorgesehen. Er bietet eine vollständige dreiachsige Messung ohne externe A/D-Wandlung. Auch der ADXL355 erfüllt dieses Integrationsprinzip; die Wahl des ADXL345 ist daher eine zu prüfende Ausgangsentscheidung und kein Nachweis seiner Überlegenheit. Reichen Auflösung oder Signal-Rausch-Abstand nicht aus, wird der ADXL355 als Alternative geprüft. Eine IEPE-Messkette ist erst bei entsprechendem Bandbreiten- oder Messqualitätsbedarf zu rechtfertigen.

Die Pilotmessung prüft effektive Erfassungsrate, neue Sensorwerte, Datenlücken, Sättigung und den Abstand des Nutzsignals zum Rauschboden bei dokumentierter Montage (A1). Die Akzeptanzgrenzen sind vor der abschließenden Testserie festzulegen. Beim ADXL345 beträgt die maximal empfohlene Sensorausgaberate bei 400-kHz-I²C 800 Hz; bei 100-kHz-I²C sind es 200 Hz. Diese Grenzen ersetzen keine Messung der tatsächlich erreichten Erfassung (Analog Devices, 2022, S. 17).

Die Edge-Plattform muss neben der Inferenz auch Datenerfassung, Vorverarbeitung, Baselines und Messwerkzeuge tragen (A4, A7, A8). Tabelle 5-5 vergleicht deshalb den

Ressourcenrahmen und die daraus abgeleiteten Integrationsfolgen. Taktraten und Beschleunigerangaben erlauben keinen unmittelbaren Vergleich der Inferenzzeiten.

Tabelle 5-5: Bewertung der Edge-Plattformen

Alternative	Dokumentierter Ressourcenrahmen	Bewertung für den Methodenvergleich
STM32H743	Cortex-M7 bis 480 MHz; bis 2 MB Flash und 1 MB RAM	Hardwarenahe Umsetzung möglich; knapper Speicher und eigenständige Portierung der Pipeline
Raspberry Pi 4 Model B	Vier Cortex-A72-Kerne mit 1,8 GHz; RAM im GB-Bereich	Plausible Linux-Plattform für CPU-Inferenz und Messwerkzeuge
Raspberry Pi 5	Vier Cortex-A76-Kerne mit 2,4 GHz; RAM im GB-Bereich	Nominell größere CPU-Reserve; konkrete Latenz und Speicherbelegung bleiben zu messen
Jetson Orin Nano Super	Sechs Cortex-A78AE-Kerne; 8 GB RAM; GPU-Beschleunigung	Sinnvoll bei GPU-bedürftigen Modellen; Nutzen für den vorgesehenen CPU-Vergleich nicht belegt

Quelle: Eigene Bewertung nach STMicroelectronics (2026), Raspberry Pi Ltd. (o. J., 2026) und NVIDIA (o. J.).

Die Ressourcenangaben werden nicht in erwartete Inferenzzeiten umgerechnet. Die gewichtete Vorauswahl verbindet den nominellen Ressourcenrahmen mit der Integration der vollständigen Pipeline. Das Gewicht für zeitliche Kontrollierbarkeit bleibt begrenzt, weil H2 ein empirisches Latenzziel und keine harte Echtzeitgarantie formuliert.

Tabelle 5-6: Kriterien und Gewichte für die Edge-Plattform

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
E1 Nomineller Ressourcenrahmen	35 %	A8. 0: keine plausible Reserve; 2: eingeschränkter Rahmen; 4: GB-Arbeitsspeicher und größere nominelle CPU-Reserve im Variantenfeld. Kein Laufzeitnachweis.
E2 Integration der Gesamtpipeline	35 %	A4–A6. 0: keine geeignete Umgebung; 2: erhebliche Anpassungen; 4: allgemeine Linux-/Python-Umgebung für Erfassung, Verfahren und Messwerkzeuge.
E3 Zeitliche Kontrollierbarkeit	15 %	A7. 0: Ablauf kaum beeinflussbar; 2: allgemeines Betriebssystem mit schwankender Last; 4: hardwarenah planbare Ausführung. Keine Fristgarantie.
E4 Passung zum CPU-Kernvergleich	15 %	FF1, FF3. 0: ungeeignete Plattformklasse; 2: zusätzliche Beschleuniger bleiben ungenutzt oder die Plattform erzwingt einen Portierungsfokus; 4: CPU-System ohne solche Zusatzschwerpunkte.

Quelle: Eigene Kriterien und Gewichte aus Kapitel 4. Ressourcen, Softwareintegration, Zeitplanung und Untersuchungsumfang werden getrennt betrachtet.

Tabelle 5-7: Gewichtete Bewertung der Edge-Plattformen

Alternative	E1	E2	E3	E4	Nutzwert
STM32H743	1	1	4	2	1,60
Raspberry Pi 4	3	4	2	4	3,35
Raspberry Pi 5	4	4	2	4	3,70
Jetson Orin Nano Super	4	4	2	2	3,40

Quelle: Eigene Punktvergabe nach dem dokumentierten Ressourcenrahmen und den Kriterien E1-E4; keine gemessenen Leistungswerte.

Der STM32 wird bei Ressourcen und Integration wegen des begrenzten Speichers und der notwendigen Portierung niedrig bewertet. Die Linux-Plattformen erhalten bei der allgemeinen Integration denselben Wert. Der Pi 4 erhält für E1 einen Punkt weniger als der Pi 5; dies bewertet ausschließlich die nominelle CPU-Reserve. Die Jetson-GPU bringt für den hier abgegrenzten CPU-Vergleich keinen zusätzlichen Untersuchungsnutzen. Ihre Leistungsfähigkeit wird damit nicht infrage gestellt.

Der Raspberry Pi 5 erreicht 3,70 Punkte und wird als Zielplattform gewählt. Er bleibt bei den acht einzelnen Gewichtsänderungen und bei gleichen Gewichten vorn. Die Einschätzung ist jedoch punktempfindlich: Wird seine Ressourcenbewertung E1 um einen Punkt gesenkt, liegt er mit 3,35 gleichauf mit dem Pi 4 und unter dem Jetson mit 3,40. Die nominelle Reserve ist deshalb eine Entwurfsbegründung, kein Leistungsbeleg. Die Erfüllung von A7 und A8 wird an der vollständigen Pipeline gemessen; ein zusätzlicher Hardwarebenchmark gehört nicht zum Kernversuch.

5.4 Datenbasis, Signalrepräsentation und Verfahren

Für die Evaluation werden eigene reale Messläufe gewählt. Damit umfasst die Untersuchung die vollständige Kette vom Sensor bis zur Entscheidung. Öffentliche Datensätze können die Methodenentwicklung unterstützen, ersetzen jedoch keinen Nachweis an der gewählten Messkette. Simulierte Daten eignen sich für Funktionstests, belegen aber keine Erkennungsleistung am realen Prüfstand. Ganze Aufnahmen werden nach A2 getrennt zugeordnet. Training und Schwellenwertkalibrierung verwenden ausschließlich Normaldaten.

Tabelle 5-8: Bewertung der Signalrepräsentationen

Darstellung	Informationsgehalt und Grenze	Vorgesehene Rolle
Dreiachsiges Zeitfenster	Erhält Reihenfolge und Achsenbeziehungen; höhere Eingabedimension	Gemeinsame Ausgangsdaten für den Hauptvergleich
Zeitbereichskennwerte	Kompakt und interpretierbar; zeitliche Struktur wird verdichtet	RMS als statistische Baseline
Frequenzspektrum mittels FFT	Zeigt Frequenzanteile; abhängig von Abtastung, Fensterdauer und Fensterfunktion	Ergänzende Analyse gespeicherter Pilotmessungen
Zeit-Frequenz-Darstellung	Zeigt zeitlich wechselnde Frequenzanteile; zusätzliche Parameter und Rechenaufwand	Keine zusätzliche Variante im Kernvergleich

Quelle: Eigene Einordnung auf Grundlage von Ruff et al. (2021) und SciPy developers (o. J. a, o. J. b).

Die Fast Fourier Transform (FFT) ist ein effizienter Algorithmus zur Berechnung der diskreten Fourier-Transformation. Sie wird ergänzend eingesetzt, um beobachtbare Frequenzanteile in Pilotmessungen zu untersuchen und die Wahl von Erfassungsrate und Fensterdauer zu unterstützen. Für H gleichmäßig abgetastete Werte beträgt der Abstand der Frequenzstützstellen $\Delta f = f_s/H$; dies allein beschreibt noch nicht die Trennbarkeit benachbarter Frequenzanteile (SciPy developers, o. J. a, o. J. b).

Eine FFT behebt weder Datenlücken noch Aliasing. Vor ihrer Auswertung müssen zeitliche Regelmäßigkeit, nutzbare Sensorbandbreite und mögliche Frequenzüberlagerungen durch die Abtastung geprüft werden. Mittelwertbehandlung, Fensterfunktion und Amplitudenskalierung sind zu dokumentieren. Die Spektralanalyse erfolgt ergänzend außerhalb der Live-Pipeline. FFT-Merkmale würden eine weitere Verfahrensvariante bilden und müssten vor der Testauswertung festgelegt sowie mit ihrem Zusatzaufwand bewertet werden. Die FFT selbst liefert keine Anomalieentscheidung.

Tabelle 5-9: Auswahl der Anomalieverfahren

Verfahren	Rolle und Auswahlgrund	Wesentliche Grenze
Autoencoder	Neuronales Hauptverfahren; prüft den Nutzen einer erlernten multivariaten Rekonstruktion	Eine gute Trennung der Zustände ist empirisch zu prüfen
RMS mit Schwelle	Statistische Minimalbaseline; transparente Verdichtung auf einen Kennwert	Verwirft die zeitliche Reihenfolge
Isolation Forest	Klassische Lernbaseline; ergänzt den RMS um eine multivariate Bewertung	Nutzt keine explizite zeitliche Nachbarschaft
One-Class SVM	Plausible Alternative zur klassischen Lernbaseline	Zusätzliche Kernel- und Hyperparameterwahl; keine weitere Vergleichsrolle
Überwachte Klassifikation	Geeignet bei repräsentativen gelabelten Fehlerklassen	Benötigt Fehlerbeispiele im Training und passt damit nicht zum gewählten Szenario

Quelle: Eigene Rollenverteilung nach Liu et al. (2008), Ruff et al. (2021) und scikit-learn developers (o. J. d).

Der Hauptvergleich umfasst Autoencoder, RMS und Isolation Forest. So werden die in Kapitel 1 festgelegten drei Verfahrensklassen abgedeckt. Alle erhalten dieselben $H \times 3$ -Ausgangsfenster. Die Standardisierung wird ausschließlich an Trainingsdaten angepasst. Der Autoencoder rekonstruiert das standardisierte Fenster, der Isolation Forest verwendet dessen Werte in fester Reihenfolge als Vektor. Die RMS-Baseline berechnet einen gemeinsamen Effektivwert über alle $3H$ standardisierten Werte; dieser Score ist dimensionslos und vom RMS der Beschleunigung in g zu unterscheiden.

Für jedes Verfahren wird die Schwelle als empirisches 99. Perzentil seiner Scores auf separaten normalen Validierungsaufnahmen bestimmt. Größere Scores bedeuten stets eine stärkere Auffälligkeit. Die genaue Quantilsberechnung, Modellparameter und Fensterbildung werden vor der Testauswertung festgehalten (A2, A3). Eine bessere Erkennungsqualität des Autoencoders wird damit nicht vorausgesetzt, sondern durch H1 geprüft.

5.5 Kriterienbasierte Bewertung der Softwarealternativen

Die Softwareauswahl betrifft den Entwicklungs- und Ausführungspfad des Autoencoders. TensorFlow/Keras und PyTorch dienen zur Modellbildung; TFLite/LiteRT, ExecuTorch und ONNX Runtime bieten Wege zur lokalen Inferenz. Der Prüfstand wurde bereits in Abschnitt

5.2 bewertet. Die nachfolgenden Kriterien betreffen die Software und werden aus A4–A8 abgeleitet.

Tabelle 5-10: Kriterienbasierter Vergleich der Softwarealternativen

Entwicklungs- und Ausführungspfad	Übergang zur Inferenz	Lokale Ausführung	Integrations- und Prüfaufwand
TensorFlow/Keras → TFLite/LiteRT	Dokumentierter Konverter für Keras-Modelle; .tflite-Datei	CPU-Inferenz mit passender Laufzeit	Modelloperationen, Paketkompatibilität und Konvertierung prüfen
PyTorch → ExecuTorch	Modellexport und Anpassung an ein Backend; .pte-Datei	Arm-CPU über geeignetes Backend, z. B. XNNPACK	Export, Backend und numerische Übereinstimmung prüfen
PyTorch → ONNX Runtime	Export in das ONNX-Austauschformat	CPU-Paket für Linux ARM64	Exporter, ONNX-Operatorversionen und Laufzeit abstimmen
TensorFlow/Keras → TensorFlow direkt	Kein Wechsel des Modellformats erforderlich	TensorFlow-CPU-Paket für Linux ARM64	Paketkompatibilität und Ressourcenbedarf der vollständigen Umgebung prüfen

Quelle: Eigene Bewertung nach Google (2026a, 2026c, 2026d), PyTorch (o. J.) und ONNX Runtime developers (o. J.). Die Zielkompatibilität des konkreten Modells ist bei jeder Alternative zu prüfen.

Die gemeinsame Voraussetzung ist, dass Modelloperationen, Python-Version und Betriebssystem auf dem Zielgerät zusammenpassen (A6). Dies ist für das konkrete Modell noch nachzuweisen. Dokumentierte ARM64-Unterstützung allein bestätigt keine beliebige Raspberry-Pi-OS-Konfiguration. Für TFLite wird ein separates Interpreter-Paket beschrieben; Modelle mit zusätzlichen TensorFlow-Operationen können eine umfangreichere Umgebung benötigen (Google, 2026c). Diese Einschränkung wird bei der Modellkonvertierung geprüft.

Tabelle 5-11: Kriterien und Gewichte für die Softwareauswahl

Kriterium	Gewicht	Bewertungsmaßstab und Anforderungsbezug
S1 Entwicklung und Inferenz trennen	30 %	A5, A8. 0: kein getrennt ladbares Modell; 2: gespeichertes Modell benötigt vollständiges Framework; 4: separates Inferenzpaket vorgesehen.
S2 Modellübergang nachvollziehen	25 %	A5, A6. 0: kein dokumentierter Weg; 2: nur teilweise dokumentiert; 4: offizieller Weg zum Speichern bzw. Exportieren und Laden.
S3 Python-Anbindung	25 %	A4, A5. 0: keine Anbindung; 2: eigene Brücke nötig; 4: offiziell dokumentierte Python-API.
S4 Linux-ARM64-CPU unterstützen	20 %	A4, A6. 0: kein dokumentierter Weg; 2: eigene Portierung nötig; 4: dokumentiertes CPU-Paket für Linux ARM64. Konkrete Versionen bleiben zu prüfen.

Quelle: Eigene Kriterien und Gewichte aus Kapitel 4. Die Punkte bewerten dokumentierte Wege, nicht deren gemessenen Ressourcenbedarf.

Tabelle 5-12: Gewichtete Bewertung der Softwarealternativen

Entwicklungs- und Ausführungspfad	S1	S2	S3	S4	Nutzwert
TensorFlow/Keras → TFLite/LiteRT	4	4	4	4	4,00
PyTorch → ExecuTorch	4	4	4	4	4,00
PyTorch → ONNX Runtime	4	4	4	4	4,00
TensorFlow/Keras → TensorFlow direkt	2	4	4	4	3,40

Quelle: Eigene Bewertung der offiziellen Dokumentation von Google (2026a, 2026c, 2026d), PyTorch (o. J.) und ONNX Runtime developers (o. J.).

Die drei separaten Inferenzwege erreichen jeweils 4,00 Punkte. Sie bieten die in S1–S4 beschriebenen Möglichkeiten. Die direkte TensorFlow-Ausführung erhält bei S1 zwei Punkte, weil dabei das vollständige Framework vorgesehen ist; dafür entfällt der Formatwechsel.

Eine hohe Punktzahl sagt nichts über spätere Latenz, Speichernutzung oder Erkennungsqualität aus. Diese Größen werden nicht mit unbelegten Schätzwerten bewertet.

Der Gleichstand bleibt bei gleichen Gewichten und bei allen acht einzelnen Gewichtsänderungen bestehen. Sinkt ein bisher mit vier Punkten bewertetes Kriterium einer Alternative um einen Punkt, verliert diese ihren Gleichstand an der Spitze. Die Matrix zeigt

daher mehrere geeignete Wege und keinen eindeutigen TensorFlow-Vorsprung. Bei gleich geeigneten Alternativen ist eine begründete Entwurfsentscheidung zulässig (NASA, 2023).

Für diese Arbeit wird aus dieser Gruppe TensorFlow/Keras mit einem Float32-TFLite-Modell gewählt. Der dokumentierte direkte Konverter für Keras-Modelle ermöglicht einen klaren Weg von der Modellbildung zur gespeicherten Inferenzdatei und deren Prüfung gegen das Ausgangsmodell (Google, 2026a). Damit wird eine konkrete Umsetzung innerhalb der gleich bewerteten Gruppe festgelegt. ExecuTorch bleibt eine gleichwertige Entwurfsalternative nach den verwendeten Maßstäben; ONNX Runtime wäre besonders bei einem gewünschten Austauschformat zweckmäßig. Ein solcher Austausch ist keine eigene Anforderung des Kernversuchs.

Im Hauptvergleich wird auf Quantisierung verzichtet. So wird zunächst keine zusätzliche Änderung der Zahlendarstellung untersucht. Die Dokumentation empfiehlt ein nicht quantisiertes TFLite-Modell als Ausgangspunkt für die Kompatibilitätsprüfung (Google, 2026b). Ob die ausgewählte Laufzeit A7 und A8 erfüllt, entscheidet erst die Messung. Auch eine kleine Modelldatei garantiert keinen kleinen Prozessspeicher. Konkrete Pakete, Versionen und ihre Einbindung werden in Kapitel 6 dokumentiert.

5.6 Systementscheidung und Übergang zur Implementierung

Die abschließende Tabelle fasst die Entscheidungen und die jeweils noch erforderlichen Nachweise zusammen. Die Nutzwerte unterstützen die Auswahl innerhalb des jeweiligen Variantenfelds. Sie sind an die offengelegten Gewichte, Punkte und Prüfvorbehalte gebunden. Die praktische Eignung bleibt Gegenstand von Implementierung und Evaluation.

Tabelle 5-13: Systementscheidungen und ausstehende Nachweise

Bereich	Auswahl	Anforderungsbezug und Nachweis
Prüfstand	Lüfter mit PWM; 3,30 Punkte	A10, A11: wiederholbare Zustände und zweiter normaler Betriebspunkt
Sensorik	ADXL345 als Ausgangsvariante; ADXL355 als Alternative	A1: Signalqualität und zeitliche Erfassung vor der Testserie prüfen
Edge-Plattform	Raspberry Pi 5; 3,70 Punkte	A4, A7, A8: vollständige Pipeline und Zeitbudget messen
Datenbasis	Eigene reale Messläufe; Normaldaten für Training und Kalibrierung	A2, A3, A9: getrennte Aufnahmen und gemeinsame Testbasis
Verfahren	Autoencoder, RMS und Isolation Forest	A3, A9: gleiche Fenster und Kalibrierungsregel; Erkennungsqualität vergleichen
FFT	Ergänzende Analyse der Pilotmessungen	A1: Frequenzanteile innerhalb der überprüften Messbandbreite untersuchen
Neuronale Software	TensorFlow/Keras → Float32-TFLite; 4,00 Punkte, Gleichstand mit zwei Alternativen	A5–A8: Konvertierung, Zielkompatibilität und Ressourcen prüfen
Bedienung und Ausgabe	Start/Stopp und gespeicherte Ergebnisse; Live-GUI optional	A4, A5, A12, A13: Datenzuordnung, geordnetes Beenden und gegebenenfalls Anzeige prüfen

Quelle: Eigene Zusammenführung der Entscheidungen aus den Abschnitten 5.2–5.5.

Kapitel 6 dokumentiert Aufbau, Software, Eignungsprüfungen und das vor der Testauswertung festgelegte Versuchsprotokoll. Entwicklungsaufnahmen bleiben explorativ; eine unabhängige Bestätigung benötigt neue Testaufnahmen gemäß Abschnitt 1.4. Kapitel 7 bewertet Anforderungen und Hypothesen. Nicht bestätigte Hypothesen werden berichtet; eine nachträgliche Anpassung an die Testdaten ist ausgeschlossen.

6 Implementierung

Dieses Kapitel beschreibt den am 9. September 2026 belegten Implementierungsstand. Die Forschungsfragen aus Kapitel 1 bestimmen die Trennung zwischen Erkennungsqualität, technischem Aufwand und veränderten Normalbedingungen. Die Regeln für Datenaufteilung und Kalibrierung aus Kapitel 2 sowie die in Kapitel 3 beschriebenen Grenzen der Übertragbarkeit werden in der Verarbeitung berücksichtigt. Die Anforderungen A1–A13 aus Kapitel 4 und die Auswahlentscheidungen aus Kapitel 5 bilden den Bezug für die folgenden Komponenten und Prüfungen. Eine implementierte Funktion wird dabei nicht mit einem abgeschlossenen experimentellen Nachweis gleichgesetzt.

6.1 Vorhandener Prüfstand und dokumentierte Bedingungen

Die Implementierung läuft direkt auf dem Rechner edgepi, einem Raspberry Pi 5 Model B Rev 1.0 mit ARM64-Architektur und Debian 13. Der ADXL345 ist über I²C-Bus 1 ansprechbar. Nach der ergänzenden Nutzerangabe vom 9. September 2026 ist der Prüfstand bereits aufgebaut und angeschlossen; Lüftersteuerung und Code haben mit diesem Aufbau zuvor funktioniert. Der Nutzer identifiziert den Lüfter als ARCTIC P12 Pro PST mit 120 mm und nennt eine vom Pi getrennte externe 12-V-Versorgung. Der ADXL345 ist nach seiner Angabe in Lüfternähe befestigt; Jumperkabel und ein Adapter verbinden die Steuerung mit GPIO18. Der in Kapitel 5 gewählte Prüfstand wird deshalb als vorhandener Aufbau weiter untersucht. Die vollständige mechanische Dokumentation steht noch aus.

Der Nutzer gab nachträglich an, dass der Lüfter während des gesamten vorherigen Arbeitsdurchlaufs ausgeschaltet war. Außerdem wurde die PWM-Vorgabe zwischenzeitlich halbiert. Ausgangs- und Zielwert einschließlich der verwendeten Skala sind noch nicht geklärt. Diese Angaben werden als nachträgliche Aussagen im Arbeitsbericht dokumentiert. Die ursprünglichen Messprotokolle behalten ihre Zustandskennzeichnungen. Aus dem ausgeschalteten Lüfter folgt ohne zeitgleiche Bestätigung von Auslauf, Montage und Umgebungsbedingungen keine kontrollierte Stillstandsreferenz.

In der ersten Fortsetzung wurden ausschließlich Zustände ausgelesen. Die damals gespeicherten 25 % bei 25 kHz belegen weder einen früheren Betriebspunkt noch ein elektrisches PWM-Signal, da die Pin-Funktion zwischen Abfragen wechselte. Später meldete der Nutzer den Start mit `pinctrl set 18 op dh` und beobachtete volle Leistung. Damit galt die zuvor angefragte Stillstandsbedingung nicht mehr. Die beobachtete volle Leistung ist keine gemessene Drehzahl oder elektrische Leistung. Der tatsächliche Zeitpunkt des Nutzerbefehls ist unbekannt. Zugleich wurde die softwareseitige Steuerung für die vereinbarten Prüfungen ausdrücklich übertragen.

Der Steueranschluss ist GPIO18 am physischen Pin 12 der Pi-Stiftleiste; der physische Pin 18 gehört dagegen zu GPIO24. Der gemeldete Nutzerbefehl wählt GPIO18 als normalen Digitalausgang auf High. Für Zwischenwerte wird der bereits konfigurierte Hardware-PWM-Kanal 2 des RP1-Controllers pwmchip0 verwendet. Dazu gehört auf GPIO18 die Funktion a3 mit der Anzeige PWM0_CHAN2. Die Vorgabe beträgt 25 kHz. Gespeicherter Tastgrad, Pin-Funktion, Signal am Lüfter und mechanische Drehzahl werden getrennt bewertet.

Vor der angekündigten Abschaltung war kein konkurrierender Erfassungs- oder Steuerprozess erkennbar. Anschließend wurden 0 % über den vorhandenen PWM-Kanal eingestellt und die PWM-Pinfunktion hergestellt. Die Rücklesung bestätigte 0 ns Tastzeit bei 40.000 ns Periode, einen aktivierten Kanal und GPIO18 in Funktion a3 mit Low-Pegel. Nach zehn Sekunden Wartezeit blieb dieser ausgelesene Zustand unverändert. Den vollständigen Stillstand bestätigte anschließend der Nutzer als Sichtbeobachtung. Die genaue Befehlsfolge und Protokollierungszeiten stehen in `steuerungsuebernahme_20260909.md` und im Journal `results/fan_control_20260909/control_events.jsonl`. Diese Zeiten dokumentieren die Protokollierung der Nutzerangaben und nicht deren tatsächlichen Ausführungszeitpunkt. Die ursprünglichen Messjournale bleiben unverändert.

Tabelle 6-1: Prüfstandangaben und noch offene Randbedingungen

Bestandteil	Belegter Stand	Noch zu dokumentieren
Zielrechner und Sensor	Raspberry Pi 5; ADXL345 über I ² C-Bus 1 geprüft	Mechanische Kopplung zwischen Sensor und Prüfstand
Mechanischer Aufbau	Laut Nutzer ARCTIC P12 Pro PST, externe 12 V, ADXL345 in Lüfternähe befestigt	Befestigungsmittel, genaue Montagefläche, Achsausrichtung, Lüfterbefestigung, Unterlage und unveränderte Montage
Lüfterzustand im vorherigen Durchlauf	Nachträgliche Angabe: ausgeschaltet	Vollständiger Stillstand, Auslaufzeit, Berührungen und Fremdschwingungen je Aufnahme
PWM-Vorgabe und Pinzuordnung	0 % bei 25 kHz zurückgelesen; GPIO18 = physischer Pin 12, Funktion a3; Stillstand visuell bestätigt; nächster Sollwert laut Nutzer 25 %	Frühere Halbierung und elektrischer Signalverlauf; der neue 25%-Betriebspunkt wurde noch nicht angefahren
Drehzahl	Nutzer nennt GPIO18 auch als Tacho; dies widerspricht der geprüften Zuordnung als Steuerausgang	Tatsächliche Steckerzuordnung, separater Tachokanal, Beschaltung, Pull-up-Spannung und Impulse je Umdrehung

PWM und Drehzahl werden als unterschiedliche Größen geführt. Die zusätzliche Angabe GPIO18 auf die Tachofrage ist wegen des belegten PWM-Ausgangs keine bestätigte Tachozuordnung. Das Herstellerdatenblatt des P12 Pro PST unterscheidet Pin 4 des

Lüftersteckers als PWM-Eingang und Pin 3 als Tachoaussgang (ARCTIC, o. J.); diese Steckernummern sind wiederum keine GPIO- oder Stiftleistennummern des Pi. Die tatsächliche Verbindung durch den vorhandenen Adapter muss daher beschrieben werden. Ein unter `hwmon` auslesbarer Drehzahlwert des Pi-Kühlers wird dem Prüflüfter nicht zugeordnet. Ohne bestätigten Tachokanal oder eine andere dokumentierte Drehzahlmessung bleibt das Drehzahlfeld leer beziehungsweise maschinenlesbar null. Diese Einschränkung betrifft die Zuordnung von Spektralanteilen zu Drehfrequenz und Harmonischen sowie die Kontrolle des Betriebspunktes nach A10 und A11.

6.2 Softwarestruktur und Datenfluss

Die Programme liegen im Repository `masterarbeit-edge-ai`. Die Sensorerfassung, Datenvorbereitung, Modellbildung und Auswertung sind in getrennten Modulen umgesetzt. Die Projektumgebung `.venv` enthält Python 3.13.5, NumPy 2.5.1, pandas 3.0.3, SciPy 1.18.0, scikit-learn 1.9.0, TensorFlow 2.21.0, `ai-edge-litert` 2.1.6 und `smbus2` 0.6.1. Diese Versionsangaben beziehen sich auf die dokumentierte Systemaufnahme des ersten Durchlaufs. Die zweite Umgebung `.venv_tf` bleibt gesondert erhalten. Laufprotokolle erfassen die tatsächlich verwendete Umgebung und die für den Lauf ermittelten Paketversionen.

Tabelle 6-2: Implementierte Komponenten der Verarbeitung

Komponente	Aufgabe	Zugeordnetes Ergebnis
fan_pwm.py	Vorhandenen Hardware-PWM-Kanal prüfen und explizite Stellbefehle ausführen	Steuerjournal, Rücklesung von PWM und Pin-Funktion
adxl345.py	Sensor konfigurieren, Register zurücklesen und neue FIFO-Einträge lesen	XYZ-Werte, Hostzeit und Qualitätsflags
collect_real_data.py	Zeitlich begrenzte Aufnahme journalisieren	CSV-Rohdaten und JSON-Begleitprotokoll
calibrate_and_train.py	Neue Normalaufnahmen trennen, skalieren und Modell bilden	Profil, Scaler, Keras-/TFLite-Modell und P99
common_comparison.py	RMS, Isolation Forest und TFLite auf denselben Fenstern ausführen	Vergleichsbundle, Einzelentscheidungen und Kennzahlen
live_pipeline.py und Liveprogramme	Erfassung von der Entscheidung entkoppeln	Rohjournal, Entscheidungsjournal, Zeit- und Statusangaben
fft_pilot.py	Gespeicherte Pilotdaten spektral untersuchen	Achsenspektren, Analysebericht und Grafik

Der Datenfluss beginnt mit einem neuen FIFO-Eintrag des Sensors. Die Erfassung speichert den Messvektor zusammen mit seiner Herkunft. Nach der aufnahmeweisen Zuordnung werden vollständige XYZ-Fenster gebildet und mit ausschließlich aus Trainingsdaten bestimmten Parametern standardisiert. Die drei Vergleichsmethoden erzeugen daraus jeweils einen Score. Die Entscheidung verwendet die zuvor aus normalen Validierungsdaten bestimmte Schwelle. Der Testpfad enthält weder Training noch eine Anpassung dieser Schwellen.

Das Modul fan_pwm.py verwendet die Python-Standardbibliothek und greift direkt auf den vorhandenen Kernel-PWM-Kanal zu. Die zusätzliche Bibliothek rpi_hardware_pwm ist dafür nicht erforderlich. Vor Zugriffen werden RP1-Gerätezuordnung, Kanal, Periode und Polarität geprüft. Ein exklusiver Prozess-Lock koordiniert Programme, die diesen Steuerweg verwenden. Beim Öffnen des Steuerkontexts wird der vorhandene Zustand gelesen; beim Schließen wird der Lock freigegeben. Beide Schritte ändern die PWM nicht. Stellbefehle erfolgen ausschließlich durch ausdrückliche Methodenaufrufe und werden mit Rücklesung protokolliert. Die bestehenden Programme live_monitor.py und live_tflite_fan_control.py verwenden diesen gemeinsamen Steuerweg über FanController. Dieser Entwicklungsadapter setzt beim Start die gewählte Vorgabe und fordert beim Beenden ausdrücklich 0 % mit aktiviertem PWM-Kanal an. Auch bei Aufnahme Fehlern wird seine Sperre freigegeben. Ein

nicht bestätigter Stellbefehl führt zu einem Fehlerstatus; daraus wird kein Stillstand abgeleitet. Die GUI wird erst am Programmeinstieg eingerichtet, damit der Aktuator ohne grafische Sitzung importierbar bleibt. Diese Softwarepfade wurden mit hardwarefreien Tests geprüft. Auf dem realen Pi wurde der neue Steuerweg bisher ausschließlich zum Lesen und Prüfen der bereits gesetzten 0-%-Vorgabe verwendet; ein Stellbefehl über dieses Modul wurde dort noch nicht ausgeführt.

6.3 Korrigierte Sensorerfassung und Rohdatenprotokoll

Die frühere Messkette konfigurierte die interne Sensorausgaberate und den Messbereich nicht ausdrücklich. Die damalige Angabe von 500 Abfragen pro Sekunde belegt deshalb keine 500 neuen Beschleunigungsmessungen pro Sekunde. Die korrigierte Implementierung setzt die Konfiguration vor einer Aufnahme und liest die betreffenden Register zurück. Bei unbekannter oder auf 100 kHz konfigurierter I²C-Busfrequenz wird höchstens eine ODR von 200 Hz zugelassen. Die technische Begründung dieser Begrenzung folgt der in Abschnitt 5.3 behandelten I²C-Empfehlung des ADXL345-Datenblatts. Die Buskonfiguration ersetzt keine elektrische Messung der Taktleitung.

Tabelle 6-3: Aktuelle Pilotparameter und ihre Aussagegrenzen

Parameter	Implementierte Vorgabe	Begründung und Prüfbedarf
Sensorausgaberate	200 Hz	Technischer Ausgangspunkt für die vorhandene I ² C-Konfiguration; relevantes Nutzband noch zu prüfen
Auflösung und Bereich	Full Resolution, ± 2 g	Register werden geprüft; ausreichende Reserve gegen Sättigung benötigt Betriebsdaten
Datenabruf	FIFO-Stream, 32 Einträge	Lesen nur bei vorhandenem Eintrag; Überlaufverdacht wird markiert
Zeitstempel	Monotone Hostzeit nach Datenburst	Zeitpunkt des Auslesens, kein sensorinterner Zeitstempel
Modellfenster	H = S = 128, drei Achsen	Aktuelle feste Eingabeform; nominell 0,64 s bei 200 Hz, Eignung noch offen
FFT-Pilot	512 Werte, 50 % Überlappung, Hann	Ausgangspunkt zur Spektralprüfung; keine automatische Festlegung endgültiger Messparameter

Ein Prozess-Lock verhindert konkurrierende Zugriffe durch die neuen Erfassungsprogramme. Fremde Programme berücksichtigen diesen Lock nicht zwangsläufig; daher wird vor Hardwareversuchen zusätzlich nach laufenden Prozessen und geöffneten Geräten gesucht.

Vor jedem XYZ-Burst werden Status und FIFO-Füllstand geprüft. Gleiche quantisierte XYZ-Tupel können auch bei neuen Sensormessungen auftreten. Der Frischwertnachweis stützt sich auf die FIFO-Folge und nicht auf eine Forderung nach unterschiedlichen Zahlenwerten.

An jeder Aufnahmegrenze werden noch nicht aufgezeichnete Vorlaufwerte bewusst verworfen und der FIFO neu gestartet. Ein voller FIFO, ein gesetzter Überlaufstatus oder eine Lesepause über der nominellen FIFO-Zeitkapazität kennzeichnen einen möglichen Datenverlust. Die exakte Zahl verlorener Sensorwerte ist daraus nicht ableitbar. Sättigung wird konservativ nahe den digitalen Messbereichsgrenzen markiert. Beim geordneten Schließen stellt die Verbindung die vorherigen Konfigurationsregister wieder her.

Das CSV-Journal enthält die drei Achsen in g, den Betrag des Messvektors, Zustandslabel, Sampleindex, relative Hostzeit, monotone Hostzeit, nominell geschätzte Sensorzeit sowie FIFO-, Verlust- und Sättigungsangaben. Die geschätzte Sensorzeit ergibt sich aus Sampleindex und nomineller ODR und ist ausdrücklich kein gemessener Sensorzeitstempel. Das JSON-Protokoll ergänzt Aufnahmepurpose, Montage, PWM-Vorgabe, gegebenenfalls unabhängig gemessene Drehzahl, Registerrücklesung, Quellcodehashes, Laufstatus und CSV-Prüfsumme.

Neue Ausgabepfade werden exklusiv angelegt. Die Erfassung erhält bei behandeltem SIGINT, SIGTERM oder Lesefehler die bereits gelesenen Werte und schreibt den Abschlussstatus. Das geordnete Ende führt Flush und fsync aus. Dies ist kein Nachweis eines verlustfreien Verhaltens bei Stromausfall oder SIGKILL. Die Anforderungen A5 und A13 werden damit technisch unterstützt; die Reichweite der geprüften Abbruchfälle bleibt ausdrücklich begrenzt.

6.4 Datenaufteilung, Modelle und Schwellen

Das Kalibrierungsprogramm ordnet vollständige Normalaufnahmen vor der Fensterbildung Training oder Validierung zu. In der aktuellen Implementierung bilden die zuerst aufgenommenen Dateien den Trainingsanteil und die abschließenden Dateien den konfigurierten Validierungsanteil. Ein Fenster überschreitet keine Aufnahmegrenze. Unvollständige Restfenster werden mit ihrer Samplezahl ausgewiesen. Die chronologische Zuordnung garantiert keine statistische Unabhängigkeit der aufeinanderfolgenden Messläufe; der tatsächliche Versuchsablauf muss zusätzlich dokumentiert werden.

Vor Datenvorbereitung und Training werden CSV- und JSON-Prüfsummen, Zustandsangaben, Sensorparameter, Zeitstempel, fortlaufende Sampleindizes und Qualitätsflags geprüft. Die Prüfung umfasst auch einen später verworfenen Fensterrest. Bereits ein markierter Gap-, Overrun- oder Sättigungsfall führt zum Zurückweisen der Aufnahme für die neue

Kalibrierung. Diese technischen Prüfungen belegen noch kein ausreichendes Nutzband oder Signal-Rausch-Verhältnis.

Der StandardScaler wird je Achse ausschließlich an den vollständigen normalen Trainingsfenstern angepasst. Seine Parameter werden anschließend unverändert verwendet. Alle Vergleichsmethoden erhalten dieselben Float32-Fenster mit H mal drei Werten. Für die FFT-Diagnose wird je Segment der Mittelwert entfernt; diese diagnostische Verarbeitung ist von der trainingsbasierten Standardisierung der Modelle zu unterscheiden.

Tabelle 6-4: Gemeinsame Eingabebasis und implementierte Verfahren

Verfahren	Eingabe und Score	Modellbildung und Schwelle
RMS	Quadratwurzel des Mittelwerts aller quadrierten standardisierten Fensterwerte; dimensionslos	Kein gelerntes Modell; lineares P99 der normalen Validierung
Isolation Forest	Dasselbe Fenster in C-Reihenfolge als Vektor; Score = -score_samples	200 Bäume, max_samples = auto, Seed 42, ein Worker; normales Training und Validierungs-P99
Float32-TFLite-Autoencoder	Standardisiertes XYZ-Fenster; mittlerer quadratischer Rekonstruktionsfehler	Normales Training in Keras; gespeicherte Validierungs-P99 bleibt nach Konvertierung unverändert

Der Autoencoder verarbeitet Fenster der Form 128×3 . Der Encoder enthält zwei eindimensionale Faltungen mit acht beziehungsweise vier Filtern und Kernelgrößen fünf beziehungsweise drei. Nach jeder Faltung halbiert Max-Pooling die zeitliche Länge. Im Decoder folgen Faltungen mit vier und acht Filtern sowie jeweils eine Verdopplung der zeitlichen Länge. Eine abschließende lineare Faltung mit drei Filtern und Kernelgröße fünf rekonstruiert die Eingangskanäle. Die übrigen Faltungen verwenden ReLU. Diese Architektur ist implementiert; ihre Eignung für die korrigierte Messkette muss durch neue Daten geprüft werden.

Die aktuelle Trainingskonfiguration verwendet Seed 42, Batchgröße 32 und höchstens 100 Epochen. ModelCheckpoint und Early Stopping überwachen den Validierungsfehler; Early Stopping verwendet eine Geduld von zehn Epochen und eine Mindestverbesserung von 10^{-6} . Dieselbe normale Validierung dient anschließend der P99-Kalibrierung. Sie ist auf Aufnahmeebene vom Training getrennt, jedoch kein zusätzlicher, von der Modellwahl abgeschirmter Kalibrierungssplit. Ein solcher zusätzlicher Split wäre vor neuen Versuchen ausdrücklich festzulegen und zu implementieren.

Die Konvertierung erzeugt ein Float32-TFLite-Modell ohne Quantisierung. Die vorhandene Konsistenzprüfung vergleicht Rekonstruktionen, Scores und Entscheidungen mit Keras auf denselben Validierungsfenstern. Im Code sind maximale absolute Abweichungen von 10^{-3}

für die Rekonstruktion und 10^{-4} für den MSE als Grenzen hinterlegt. Ein neues Modell benötigt eine neue Prüfung; ein erfolgreicher Vergleich alter Modelle wird nicht übertragen. Die Interpretation der Grenzen und etwaiger Entscheidungsabweichungen ist im jeweiligen Konvertierungsbericht enthalten.

Das gemeinsame Bundle bindet Modelle, Scaler, Profilkopie, Datenzuordnung und Quellcode über SHA-256. Alle Scores sind so ausgerichtet, dass ein größerer Wert stärker auffällig ist. Ein Score oberhalb seiner Schwelle ergibt ANOMALY; andernfalls ergibt ein gültiger Score NORMAL. Nichtendliche Werte oder unzulässige Modellausgaben führen zu INVALID. Die P99-Regel ist für alle Methoden gleich, die numerischen Schwellen sind methodenspezifisch. Validierungsüberschreitungen sind Kalibrierungsdiagnosen und keine unabhängige Fehlalarmmessung.

6.5 Liveverarbeitung und Ressourceninstrumentierung

Die Livepipeline erfasst Sensorwerte in einem getrennten Thread und persistiert die Rohdaten. Eine Warteschlange mit Platz für vier vollständige Fenster entkoppelt die Erfassung von der Entscheidung. Bei Überlast werden verlorene Entscheidungsfenster gezählt; sie werden nicht als verarbeitete Normalfenster ausgegeben. Die Rohdatenpersistenz und der geordnete Abschluss sind getrennt von der rechtzeitigen Verarbeitung jedes Fensters zu beurteilen.

Die Verarbeitungslatenz beginnt mit dem vollständigen Fenster und endet bei der verfügbaren Entscheidung. Wartezeit, Vorverarbeitung und Inferenz werden getrennt instrumentiert. CPU-Zeit und RSS beziehen sich auf den gesamten jeweiligen Prozess. Die Bezugsgröße von 100 % CPU ist ein Kern; zusätzlich ist eine Normierung auf alle logischen Kerne möglich. Bibliotheken und Vorverarbeitung gehören zum Prozessspeicher. Die gespeicherte Modelldateigröße wird als eigene Größe geführt.

Ein Ressourcen-Replay arbeitet ohne Sensor- und Fensterfüllwartezeit. Es eignet sich zum Vergleich der Verarbeitung derselben gespeicherten Fenster, ersetzt aber keine Liveprüfung von A7 und A8. Für H2 werden zusätzlich Fristüberschreitungen und Erfassungspausen benötigt. Die optionale Oberfläche verwendet begrenzte Nachrichtenpuffer und kennzeichnet veraltete Daten. Eine sichtbare Funktionsprüfung und eine kontrollierte Messung ihrer Zusatzlast stehen aus, da im bisherigen Terminal keine nutzbare Desktop-Anmeldung bestätigt wurde.

Die vorhandene Lüfterabschaltung ist ein gesonderter Entwicklungsmodus. Ihr RUN-Zustand verwendet 100 % Ansteuerung. Dieser Modus wird für den Methodenvergleich nicht verwendet, weil eine detektionsabhängige Abschaltung die anschließend erfassten Daten verändern würde. Die Messprogramme dokumentieren eine separat eingestellte PWM-

Vorgabe, ohne dadurch einen Ausgang zu setzen. Seit der ausdrücklichen Übertragung übernimmt die Softwaresteuerung die angekündigten Zustandswechsel des Prüfstandes. Zwischenwerte werden über den vorhandenen Hardware-PWM-Kanal eingestellt. Konkreter Tastgrad, Einlaufphase und Aufnahmebeginn werden vor dem jeweiligen Versuch benannt. Während der Aufnahme bleibt der Steuerkontext geöffnet und hält seinen Lock; Sollwert und Pin-Funktion können ohne Stellbefehl zurückgelesen werden. Das Ende dieses Kontexts schaltet den Lüfter nicht automatisch um. Es erfolgen keine erkenntnisabhängigen Zustandsänderungen. Direkte pinctrl- oder sysfs-Zugriffe fremder Programme beachten den Lock nicht zwangsläufig. Vor jedem Hardwarelauf werden deshalb konkurrierende Steuerungs- und Erfassungsprozesse erneut geprüft; eine einmalige Prozessprüfung schließt spätere Fremdzugriffe nicht aus.

6.6 Technische Prüfbelege und deren Grenzen

Die vorhandenen Belege sind unter results/verification_20260909 und results/common_comparison_20260909 abgelegt. Sie dokumentieren Softwarekorrekturen und technische Entwicklungsversuche. Die erneute Bewertung der früheren Aufnahmen verändert diese Dateien nicht. Tabelle 6-5 trennt den jeweiligen Befund von einem noch fehlenden experimentellen Nachweis.

Tabelle 6-5: Vorhandene Entwicklungsbelege und offene Nachweise

Prüfung	Nachgewiesener Umfang	Aussagegrenze
Software-Regression des ersten Durchlaufs	61 Tests und zwölf Untertests bestanden; Bericht <code>software_tests_final_v2.xml</code>	Keine mechanische Zustands- oder SNR-Bestätigung
FIFO-Sensorpilot	Neue FIFO-Einträge und Registerkonfiguration; im gespeicherten Pilot keine gesetzten Qualitätsflags	Damalige Randbedingungen nicht kontrolliert protokolliert
Fehler- und Abbruchproben	Absichtliche Lesepause als Überlauf erkannt; Teilaufnahme nach SIGTERM erhalten	Kein Stromausfalltest; keine genaue Verlustzählung
TFLite und Vergleichspipeline	Alte Profilartefakte reproduziert; gemeinsamer Validierungsvergleich vorbereitet	Keine unabhängige Testgüte der korrigierten Messkette
Kurzer Livepilot und Ressourcen-Replay	Technische Ausführung und Zeitinstrumentierung auf dem Pi geprüft	Kein abschließender Langzeit-, GUI- oder H2-Nachweis
Übernahme der Lüftersteuerung	GPIO18/physischer Pin 12 und RP1-Kanal geprüft; 0 % bei 25 kHz mit Funktion <code>a3</code> gesetzt; Stillstand danach vom Nutzer visuell bestätigt	Keine gemessene Drehzahl oder elektrische Signalform; Montage und Tachozuordnung noch zu klären
Wiederverwendbarer PWM-Steuerweg	28 Backend- und 17 Adapterprüfungen bestanden; vorhandene 0%-Vorgabe mit neuem Modul auf dem Pi ausschließlich lesend geprüft	Kein realer Stellbefehl mit dem neuen Modul; Testnachweis ersetzt keine Signal- oder Drehzahlmessung

Die abschließende Regression nach Integration der PWM-Steuerung umfasst 106 bestandene Tests und zwölf Untertests. Der Bericht liegt unter `results/fan_control_20260909/software_tests_final_v3.xml`. Die neuen Steuerungsprüfungen verwenden temporäre Dateisystemmodelle und nachgebildete Pinrückmeldungen; sie erzeugen am Prüfstand kein PWM-Signal.

Der frühere Livepilot erzeugte mit alten Modellartefakten und ausdrücklich zugelassener Messratenabweichung ausschließlich ANOMALY-Entscheidungen. Diese Entscheidungen belegen weder Fehlererkennung noch Fehlalarme: Die Referenzmodelle gehörten zur früheren Messkette, und eine kontrollierte Zustandsreferenz fehlte. Auch die nachträgliche Ausschaltangabe beseitigt diese Einschränkungen nicht. H1–H3 bleiben bis zu den jeweils vorgesehenen unabhängigen Messungen unbestätigt.

6.7 Kontrollierte Pilotfolge und Vorbereitung der Evaluation

Die nächste Messung beginnt erst bei vollständig stillstehendem Lüfter und geklärt, unveränderter Montage. Der Nutzer hat den Stillstand nach der softwareseitigen 0%-Vorgabe visuell bestätigt; konkrete Befestigung und Steckerzuordnung sind noch offen. Die geplante 30-s-Stillstandsaufnahme verwendet die korrigierte FIFO-Erfassung und erhält den Zweck *pilot*. Sie bildet eine Hintergrundreferenz für die Messkettenprüfung und wird nicht als Normaltraining des laufenden Lüfters verwendet. Anschließend wird die Betriebsmessung mit dem vom Nutzer ausdrücklich gewünschten Sollwert von 25 % und bestätigter Bereitschaft angekündigt. Dieser neu festgelegte Sollwert ist keine Ableitung aus der früheren Halbierung. Das Einschalten und Einstellen übernimmt die Softwaresteuerung. Als vorläufige Einlaufphase sind 30 s vorgesehen; die verstrichene Zeit allein beweist keinen stabilen Lauf. Während jeder Aufnahme bleiben Montage und Betriebspunkt unverändert. Der neue Betriebspunkt wurde bis zum Stand dieses Kapitels noch nicht angefahren.

Stillstands- und Betriebsdaten werden auf Mittelwerte, AC-RMS, Wertebereiche, Frequenzanteile, beobachteten FIFO-Durchsatz, Hostzeitlücken und Qualitätsflags untersucht. Die FFT-Diagnose verwendet Hann-Fenster, Mittelwertentfernung und einseitige Leistungsdichten. Ihre Frequenzachse beruht auf dem beobachteten FIFO-Durchsatz unter der Annahme eines hinreichend stabilen Sensortaktes ohne unerkannten Verlust. Eine Spektralspitze wird ohne unabhängige Drehzahlmessung nicht als Drehfrequenz bezeichnet. Ein geringer Anteil nahe der Bandgrenze beweist keine Aliasingfreiheit.

Der Leistungsabstand zwischen Betrieb und Stillstand wird für ein dokumentiertes gemeinsames Frequenzband bestimmt. Das Verhältnis $10 \log_{10}(P_B/P_S)$ beschreibt Betriebs- zu Stillstandsleistung. Die Stillstandsleistung enthält neben Sensorrauschen auch Umgebungsanregungen. Erst unter der zusätzlichen Annahme unveränderter, additiver und unkorrelierter Hintergrundanteile darf aus $P_B - P_S$ bei positivem Unterschied ein Überschuss-SNR geschätzt werden. Eine solche Schätzung wird mit ihren Annahmen ausgewiesen und nicht als isolierte Sensoreigenrauschmessung bezeichnet.

Die endgültige Auswahl von ODR, Messbereich, Frequenzband, H/S und Aufnahmedauer erfolgt vor neuer Modellbildung anhand dieser Piloten. Null gesetzte Gap-, Overrun- und Sättigungsflags sind bereits technische Freigabekriterien. Weitere numerische Grenzen für Signaltrennung, Ratenschwankung und Drift sind noch offen. Sie müssen begründet und vor der abschließenden Testserie festgehalten werden. Falls Bandbreite oder Signalqualität nicht ausreichen, bleibt A1 offen. Ein erfolgreiches Training würde diesen fehlenden Nachweis nicht ersetzen.

Der vorbereitete Versuchsplan sieht zunächst sechs normale Trainings- und zwei normale Validierungsaufnahmen mit jeweils 60 s als Vorschlag vor. Diese Planwerte sind keine bereits erhobenen Daten und werden nach der Pilotprüfung festgelegt. Aufnahme und Training können mit record-only und train-only getrennt ausgeführt werden. Erst danach wird das gemeinsame RMS-/Isolation-Forest-/TFLite-Bundle erzeugt und vor den unabhängigen Tests eingefroren. Konkrete Anomaliezustände, ihre reproduzierbare Herstellung und Rücksetzung sowie Zahl und Reihenfolge der Testwiederholungen sind vor der Testphase zu dokumentieren.

Für den zweiten normalen Betriebspunkt werden dieselben Modelle, dieselbe Skalierung und sämtliche Schwellen unverändert verwendet. Er bleibt normale Klasse und erhält eine eigene Zustandskennung. Ein separat trainiertes zweites Profil beantwortet FF4 nicht. Die späteren Kennzahlen werden insgesamt, je Zustand und je Aufnahme ausgewertet; Fenster derselben Aufnahme gelten nicht als unabhängige Wiederholungen. Kapitel 7 erhält Evaluationsergebnisse erst nach den entsprechenden Messungen. Der detaillierte Ablauf und die offenen Entscheidungen sind in `versuchsplan_kontrolliert_20260909.md` dokumentiert.

7 Evaluation und Ergebnisse

8 Diskussion

9 Ausblick

10 Fazit

11 Literaturverzeichnis

- Analog Devices. (2022, Mai). ADXL345: 3-axis, ± 2 g/ ± 4 g/ ± 8 g/ ± 16 g digital accelerometer (Rev. G) [Datenblatt]. [Datenblatt ADXL345](#)
- Analog Devices. (2025, Juni). ADXL354/ADXL355: Low noise, low drift, low power, 3-axis MEMS accelerometers (Rev. D) [Datenblatt]. [Datenblatt ADXL354/ADXL355](#)
- Antonini, M., Pincheira, M., Vecchio, M., & Antonelli, F. (2023). An adaptable and unsupervised TinyML anomaly detection system for extreme industrial environments. *Sensors*, 23(4), 2344. <https://doi.org/10.3390/s23042344>
- ARCTIC. (o. J.). P12 Pro PST: ARCTIC Specifications [Datenblatt]. Abgerufen am 7. September 2026. [Spezifikationen P12 Pro PST](#)
- Buttazzo, G. C. (2011). *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications* (3. Aufl.). Springer. <https://doi.org/10.1007/978-1-4614-0676-1>
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), Artikel 15, 1–58. <https://doi.org/10.1145/1541880.1541882>
- Department for Communities and Local Government. (2009, Januar). *Multi-criteria analysis: a manual*. ISBN 978-1-4098-1023-0. [Handbuch zur multikriteriellen Entscheidungsanalyse](#)
- Garay, C. E., Miranda Bonomi, F. A., Mansilla, G. N., Fagre, M., Guzmán, S. G., Ritorto, P. A., Perez, F. I., & Katz, M. (2026). A multimodal TinyML-based predictive maintenance architecture for industrial IoT in the 6G era. *Sensors*, 26(14), 4536. <https://doi.org/10.3390/s26144536>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Kapitel 14: Autoencoders. <https://www.deeplearningbook.org/contents/autoencoders.html>
- Google. (2026a, 28. Mai). Convert TensorFlow models. Google AI Edge, LiteRT-Dokumentation. Abgerufen am 7. September 2026. [Modellkonvertierung](#)
- Google. (2026b, 28. Mai). Post-training quantization. Google AI Edge, LiteRT-Dokumentation. Abgerufen am 7. September 2026. [Konvertierung und Quantisierung](#)
- Google. (2026c, 28. Mai). Quickstart for Linux-based devices with Python. Google AI Edge, LiteRT-Dokumentation. Abgerufen am 8. September 2026. [Separater Interpreter und Linux-Plattformen](#)
- Google. (2026d, 12. März). Install TensorFlow with pip. TensorFlow-Dokumentation. Abgerufen am 8. September 2026. [TensorFlow-Installation und Plattformunterstützung](#)
- Jardine, A. K. S., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7), 1483–1510. <https://doi.org/10.1016/j.ymssp.2005.09.012>
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), Artikel 15, 1–21. <https://doi.org/10.1145/2382577.2382579>

-
- Knap, P., Jachymczyk, U., & Lalik, K. (2026). Leakage-safe, reproducible benchmarking for vibration-based fault diagnosis. PHM Society European Conference, 9(1), 1–8. <https://doi.org/10.36001/phme.2026.v9i1.4924>
- Lessmeier, C., Kimotho, J. K., Zimmer, D., & Sextro, W. (2016). Condition monitoring of bearing damage in electromechanical drive systems by using motor current signals of electric motors: A benchmark data set for data-driven classification. PHM Society European Conference, 3(1). <https://doi.org/10.36001/phme.2016.v3i1.1577>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. In 2008 Eighth IEEE International Conference on Data Mining (S. 413–422). IEEE. <https://doi.org/10.1109/ICDM.2008.17>
- NASA. (2023, 26. Juli). 6.8 Decision analysis. In Systems Engineering Handbook. Abgerufen am 8. September 2026. [Kriterien, Entscheidungsbewertung und Unsicherheit](#)
- NVIDIA. (o. J.). Jetson Orin Nano Super Developer Kit. Abgerufen am 8. September 2026. [Technische Daten des Jetson Orin Nano Super](#)
- ONNX Runtime developers. (o. J.). Python. ONNX Runtime-Dokumentation. Abgerufen am 7. September 2026. [ONNX-Export und lokale Inferenz](#)
- Oriental Motor. (o. J.). Servo motor glossary of terms. Abgerufen am 7. September 2026. [Servoantriebe, Rückführung und Regelung](#)
- PCB Piezotronics. (o. J.). Model 356A32: Triaxial accelerometer. Abgerufen am 8. September 2026. [Spezifikationen des Beschleunigungssensors 356A32](#)
- Pittino, F., Puggl, M., Moldaschl, T., & Hirschl, C. (2020). Automatic anomaly detection on in-production manufacturing machines using statistical learning methods. Sensors, 20(8), 2344. <https://doi.org/10.3390/s20082344>
- PyTorch. (o. J.). Getting started with ExecuTorch. Dokumentation, Version 1.3. Abgerufen am 7. September 2026. [Modellexport und Ausführung mit ExecuTorch](#)
- Randall, R. B., & Antoni, J. (2011). Rolling element bearing diagnostics—A tutorial. Mechanical Systems and Signal Processing, 25(2), 485–520. <https://doi.org/10.1016/j.ymssp.2010.07.017>
- Raspberry Pi Ltd. (2026, April). Raspberry Pi 5 [Product brief]. [Produktbeschreibung Raspberry Pi 5](#)
- Raspberry Pi Ltd. (o. J.). Raspberry Pi 4 Model B specifications. Abgerufen am 7. September 2026. [Spezifikationen Raspberry Pi 4](#)
- Ren, H., Anicic, D., & Runkler, T. A. (2021). TinyOL: TinyML with online-learning on microcontrollers. In 2021 International Joint Conference on Neural Networks (IJCNN) (S. 1–8). IEEE. <https://doi.org/10.1109/IJCNN52387.2021.9533927>
- Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G., & Müller, K.-R. (2021). A unifying review of deep and shallow anomaly detection. Proceedings of the IEEE, 109(5), 756–795. <https://doi.org/10.1109/JPROC.2021.3052449>

-
- scikit-learn developers. (o. J. a). Common pitfalls and recommended practices. Dokumentation, Version 1.9. Abgerufen am 7. September 2026. [Vorverarbeitung und Data Leakage](#)
- scikit-learn developers. (o. J. b). Cross-validation: evaluating estimator performance. Dokumentation, Version 1.9. Abgerufen am 7. September 2026. [Datenaufteilung und Gruppen](#)
- scikit-learn developers. (o. J. c). Metrics and scoring: quantifying the quality of predictions. Dokumentation, Version 1.9. Abgerufen am 7. September 2026. [Definitionen der Bewertungskennzahlen](#)
- scikit-learn developers. (o. J. d). Novelty and outlier detection. Dokumentation, Version 1.9. Abgerufen am 7. September 2026. [Isolation Forest und One-Class SVM](#)
- SciPy developers. (o. J. a). Discrete Fourier transforms (scipy.fft). SciPy-Dokumentation, Version 1.18.0. Abgerufen am 7. September 2026. [Diskrete Fourier-Transformation und FFT](#)
- SciPy developers. (o. J. b). Signal processing (scipy.signal): Spectral analysis. SciPy-Dokumentation, Version 1.18.0. Abgerufen am 7. September 2026. [Spektralanalyse und Fensterfunktionen](#)
- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. IEEE Internet of Things Journal, 3(5), 637–646. <https://doi.org/10.1109/JIOT.2016.2579198>
- STMicroelectronics. (2026, Januar). STM32H742xl/G STM32H743xl/G (DS12110, Rev. 11) [Datenblatt]. [Datenblatt STM32H743](#)
- Universität Paderborn. (o. J.). Test rig and measurement equipment. Lehrstuhl für Konstruktions- und Antriebstechnik. Abgerufen am 7. September 2026. [Lagerprüfstand und Messtechnik](#)

12 Anhang